Microcomputer Systems 2

Jil Zerndt FS26
Lecture contents

01 Introduction: Embedded Linux & Yocto

Embedded Linux Basics

Embedded Linux Embedded Linux is the use of the **Linux Kernel** together with various **Open-Source components** in an embedded system.
- Kernel + userspace libraries/tools, tailored to a device
- Huge ecosystem of ready-made components

Why Linux in Embedded?
- **Re-use** of components: network stacks, multimedia, graphics, crypto libs already exist → focus on the **added value** of your product
- No need to re-write a kernel, TCP/IP-, USB-stack or GUI toolkit
- **Low cost**: free software, copy onto unlimited devices, no license fee (except GPL obligations → see L09); even the dev tools are free

Build Systems

Build System A build system automates building a complete target system (kernel + rootfs).

With build system
- Auto download, configure, compile, install in the right order
- Resolves all dependencies automatically
- Build is **reproducible** → easy to reconfigure, upgrade, fix bugs

Without build system
- Every app built by hand, rootfs from scratch
- Each config + dependency done manually
- Integrating software from different teams is painful

Available Build Systems
- **Buildroot** – community, simple, small devices
- **PTXdist** – by Pengutronix
- **OpenWRT** – Buildroot fork, network devices/routers
- **OpenEmbedded**-based: **Poky** (Yocto), Ångström

Comparison
- **Buildroot**: simple, small devices; weak for advanced features / multi-machine support
- **OpenWRT**: based on Buildroot, embedded network devices
- **Poky**: complex systems, lots of customization, **multiple targets** at once → used in this course

The Yocto Project

What is Yocto? A set of **templates, tools and methods** to build **custom** embedded Linux-based systems.
- Open-source, started by the Linux Foundation (2010), led by Richard Purdie
- Based on the **OpenEmbedded** project
- Supports x86, ARM, MIPS, PowerPC
- **Layer architecture** → easy code re-use

Yocto is NOT a distribution Yocto is a **build system**, not an embedded Linux distribution and not the Linux kernel. It **creates a custom distribution** for you.

The 3 Core Components
1. **Poky** – the reference system; collection of projects/tools to generate a new Yocto-based distribution
2. **Layers (Metadata)** – sets of recipes, layers and classes shared between OpenEmbedded-based systems
3. **BitBake** – the build engine (Python): reads/interprets the metadata (recipes), then downloads, configures and builds packages & images

Exam Quiz: Which statements about Yocto are true? (slide 14) Select one or more (multiple choice):
1. Is a Linux build system
2. Is an embedded Linux distribution
3. Yocto is based on OpenEmbedded
4. Is the Linux kernel

Correct: (1) and (3).
- (1) **true** – Yocto is a build system
- (2) false – it **creates** a distribution, it is not one
- (3) **true** – built on OpenEmbedded
- (4) false – it is not the kernel

Conceptual Yocto Questions **Map every claim to one of the 3 building blocks.** Decide whether the statement describes the **tooling/process** (build system, BitBake, Poky, layers, OpenEmbedded) or a **result** (a distribution, the kernel, an image).
- Process/tooling words → usually **true**
- ïs a distribution/ ïs the kernel"→ **false** (those are **outputs**, not what Yocto **is**)

Concept & Directory Structure

Poky Folder (downloaded) Downloaded from Yocto, left **unchanged**:
- Tools (BitBake, etc.)
- Default layer and recipes
- Layers: `meta` (oe-core), `meta-yocto`, `meta-yocto-bsp`

Additional Layers Downloaded from other sources or created by you:
- RPI layer (`meta-raspberrypi`)
- MC2 layer (`meta-mc2`)

Build Folder (generated) Generated by Yocto, configuration edited **manually**. Created/entered with `oe-init-build-env`; Poky stays untouched.
- `conf/` – image- & layer configuration
- `downloads/` – all downloaded source files
- `sstate-cache/` – shared state cache
- `tmp/` – intermediate & final files:
  - `tmp/deploy/` final output, `.../images/` flash images
  - `tmp/work/` per-architecture work dirs
  - `tmp/sysroots/` shared libs & headers for compiling
  - `tmp/buildstats/` build statistics

Clean state:

```
rm -rf tmp
bitbake -c cleansstate core-image-minimal
```

Configuring the System

The 2 Config Files Configure the system in two files inside `build/conf/`:
- `bblayers.conf` – which layers are used
- `local.conf` – target machine, directories, mirrors/repos, extra settings, additional applications

BitBake Variable Assignment (slide 26)
- `=` assign; value expanded **when used** (lazy)
- `:=` immediately expand the value
- `?=` assign **only if** nothing was assigned yet (default)
- `??=` same, but **lower** precedence (weak default)

Last non-conditional = wins; ?= loses to a real =.

bblayers.conf + machine (slides 25–28)

```
# build/conf/bblayers.conf
BBLAYERS ?= " \
  /home/mc2/poky-mickledore/meta \
  /home/mc2/poky-mickledore/meta-poky \
  /home/mc2/poky-mickledore/meta-yocto-bsp \
  /home/mc2/rpi64/meta-raspberrypi \
  /home/mc2/rpi64/meta-mc2 \
  /home/mc2/rpi64/meta-student \
"
# build/conf/local.conf
MACHINE = "raspberrypi4-64"
```

Machine file lives in `meta-<bsp>/conf/machine/<MACHINE>.conf`, e.g. `raspberrypi4-64.conf`; its name must match the `MACHINE` value.

BitBake & Building

BitBake Commands & Targets `bitbake [options] [target]` builds a target. Useful targets:

- `core-image-minimal` – small bootable CLI image
- `core-image-sato` – image with Sato (GNOME mobile UI)
- `core-image-full-cmdline` – the **MC2 image**
- `meta-toolchain` – headers/libs to develop on target
- `meta-ide-support` – cross-toolchain for the SDK

Build a Yocto Image **Step 1 – enter build env** (sets all env vars; `build/` already prepared for MC2):

```
source ~/poky-mickledore/oe-init-build-env ~/rpi64/build
```

Step 2 – configure `bblayers.conf` (layers) and `local.conf` (set `MACHINE`).
Step 3 – build:

```
bitbake core-image-full-cmdline
```

First build takes 3–4 h (downloads all sources + compiles). Output lands in `tmp/deploy/images/<machine>/`.

Layers

Create & Add a New Layer **Step 1 – create** the layer (adds config dir + optional example recipe):

```
source ~/poky-mickledore/oe-init-build-env ~/rpi64/build
bitbake-layers create-layer ../meta-student
bitbake-layers add-layer    ../meta-student
```

Step 2 – (alternative) register manually by adding the layer's **absolute path** to `BBLAYERS` in `build/conf/bblayers.conf`.

conclusion

BitBake parses every layer in `BBLAYERS` and picks up its recipes, config files and classes. Layer location on disk does not matter (but keep it next to the other layers).

Files in a Created Layer

- `conf/layer.conf` – layer config (rarely edited)
- `COPYING.MIT` – layer license (default MIT)
- `README` – description
- `recipes-*/*/*.bb` – auto-parsed by BitBake

BSP & Useful Layers **BSP layer** = metadata for specific hardware (kernel, bootloader, machine `.conf`), named `meta-<bsp>`; behaves like a normal layer.

- SoC: `meta-ti`, `meta-fsl-arm`, `meta-raspberrypi`
- `meta-qt5`, `meta-realtime`, `meta-gstreamer10`, `meta-filesystems`, `meta-browser`, ...

Recipes

Recipe A recipe is the **"list of ingredients" + "cooking instructions"** to build one or more packages. It holds configuration variables and a common set of tasks, all run through BitBake.

A .bb Recipe

Header

- `DESCRIPTION`, `HOMEPAGE`, `SECTION`
- `LICENSE`, `LIC_FILES_CHKSUM`, `COMPATIBLE_MACHINE`

Sources

- `SRC_URI` – location of source + license files
- `S` – set when source is not from an external repo

Tasks `do_compile()`, `do_install()`, ...

Recipe: hello_1.0.bb (slide 41)

```
DESCRIPTION = "Example recipe for an application"
SECTION = "examples"
LICENSE = "GPL-2"
SRC_URI = "file://hello.c file://GPL-2"
LIC_FILES_CHKSUM = "file://GPL-2;md5=94d55d...19f"
S = "${WORKDIR}"
do_compile() {
${CC} ${CFLAGS} ${LDFLAGS} ${WORKDIR}/hello.c -o hello
}
do_install() {
install -d ${D}${bindir}
install -m 0755 hello ${D}${bindir}
}
```

Default Recipe Tasks

- `do_fetch` get sources
- `do_unpack` unpack
- `do_patch` apply patches
- `do_configure`
- `do_compile`
- `do_install`
- `do_package`
- `do_rootfs` build rootfs

Override in the `.bb`; list with `bitbake <recipe> -c listtasks`.

License Checksum `LIC_FILES_CHKSUM` tracks the license file. If the upstream license changes, the checksum changes and the **build fails** until the recipe is updated. Mandatory in every recipe **unless** `LICENSE = CLOSED"`.

Cookbooks (KR)

Add an Application (slide 44) **Step 1** – (optional) create a meta-layer (`bitbake-layers create-layer new-layer`) and create the app's directory structure inside the meta-layer.
Step 2 – add the layer in `build/conf/bblayers.conf`.
Step 3 – write the application + add a license file, then create the recipe (e.g. `myexample_0.1.bb`).
Step 4 – edit `build/conf/local.conf` to install the app:

```
IMAGE_INSTALL_append = " myexample"
```

Create the SD-Card (slide 45) The SD-card (MBR) is split into two partitions:

- **BOOT** – boot loader, Linux kernel, device tree
- **ROOT** – filesystem: executables, libraries, data

Copy the generated image with `bmaptool`:

```
sudo bmaptool copy \
  tmp/deploy/images/raspberrypi4-64/\
core-image-full-cmdline-raspberrypi4-64.wic.bz2 \
  "/dev/${SD_CARD}"
```

Sample Exam (2019)

Exam: Yocto (Task 1)

- **What is Yocto?** A set of templates, tools & methods to build custom embedded Linux-based systems
- add a layer `mylayer` → register it in `build/conf/bblayers.conf`
- add an app `myapp` → register it in `build/conf/local.conf`

Exam Exercise (Probeprüfung 2021)

Exam 2021 A1: Yocto basics

1. What is Yocto?
2. In which config file do you register a new layer `mylayer`? And a new application `myapp`?

- Yocto = a set of **templates / tools / methods** to build custom embedded Linux systems
- layer → `build/conf/bblayers.conf`
- app → `build/conf/local.conf`

02 Embedded Linux System

Basic Linux Commands

Navigation & Search

- `ls -altrh` – long, all, by time, human sizes
- `cd /usr/bin`, `cd ..`, `cd ~` (home)
- `pwd` – print working dir
- `which ls` – show path of a command
- `locate stdio.h` – find file by name (db)
- `grep -iR test *` – search in files (i=case-insens., R=recursive)
- `find . -name ttest.c"` – find file in tree

Files & Permissions

- `cat` / `more` / `less` – display a file
- `cp foo bar`, `cp -a dir1 dir2` (all/recursive)
- `mv foo bar` – move / rename
- `mkdir foo`, `rm foo`, `rm -rf dir` (force, recursive)
- `chown -R foo:bar /home/foo` – change owner
- `chgrp bar /home/foo` – change group
- `df -h` (disk usage), `du -skh ~/` (space used)

Archives / Mount / Pipes / Links

- `tar zcvf l.tgz l` – compress dir (z=gzip, c=create, v=verbose, f=file)
- `tar zxvf l.tgz` – extract (x)
- `gzip -9 f.txt` / `gunzip f.gz`
- `touch foo` – empty file / update timestamp
- `mount /dev/sda1 /media/hd`, `umount /media/hd`
- Pipe/redirect: `cal > foo`, `cat < /etc/passwd`
- `cat /dev/zero > foo` – writes zeros (fills disk!)
- Backtick: `echo "date is ‘date‘"` – insert cmd output
- `ln a b` – **hard link** (2 names → same inode, survives delete, no cross-FS)
- `ln -s a b` – **soft link** (shortcut, can cross FS, breaks if target deleted)

Embedded Hardware & Architecture

Linux System Structure

Clear separation of **Userspace** and **Kernelspace**.

- Userspace: applications + C library
- Kernel: task/memory mgmt, networking, device drivers, filesystems

Boot Sequence

1. **Bootloader** – loads device tree + kernel to RAM, starts kernel
2. **Kernel** – inits hardware/subsystems, mounts root FS (from `root=`), starts init app (`/sbin/init`)
3. **/sbin/init** – starts userspace services & shell

Supported Architectures

32- and 64-bit:

- x86 / x86_64, **ARM** (100s of SoCs), RISC-V
- PowerPC (real-time/industrial), MIPS (networking)
- SuperH, Blackfin (DSP), Microblaze (Xilinx FPGA soft-core), ...

Requirements

- MMU **and** non-MMU (latter has limits)
- RAM: min 8 MB, normally $\geq$ 32 MB
- Storage: 4 MB Flash/Ramdisk; SD/MMC block storage ok
- **Not** for small microcontrollers

Hardware Platform Types

- **Component on Module (CoM)**: small board, only CPU/RAM/flash + connectors → write/integrate peripheral drivers in kernel
- **Evaluation platform**: from SoC vendor, expensive, many built-in peripherals, unsuitable for real products
- **Custom platform**: own schematics → write/integrate drivers

Filesystem

Root Filesystem

- Root of the hierarchy is /; `mount` attaches other FS inside it
- Root FS = **first mounted** FS, mounted directly by the kernel via the `root=` kernel option
- No root FS → **kernel panic**: **"VFS: Unable to mount root fs on unknown block(0,0)"** → append a correct `root=`

Exam Quiz: What is true about the root filesystem? (slide 11) Select one or more:

1. `cd ~/yocto` brings us to a `yocto` dir in root
2. Under root are: `~/yocto` or `~/board`
3. Under root are: `/etc` or `/home`
4. `cd /sys` brings us to a `sys` dir in root

Correct: (3) and (4).

- (1),(2) false – `~` is the **home** dir (`/home/<user>`), **not** root /
- (3),(4) true – `/etc`, `/home`, `/sys` sit directly under /

Filesystem Path Questions

Decide where a path points by its **first character**:

- starts with / → absolute, under **root**
- starts with ~ → under **home** (`/home/<user>`), never directly under root
- starts with `./` or a name → relative to current dir

Core Directories

- `/proc` – proc pseudo-FS (OS/process info)
- `/sys` – sysfs pseudo-FS (devices/buses)
- `/dev` – device files
- `/lib` – basic libraries & modules
- `/bin`, `/sbin` – basic (system) programs
- `/etc` – system-wide configuration
- `/media`, `/mnt` – removable / static mount points

Additional Directories

- `/boot` – kernel image (on PCs)
- `/home` – user home dirs, `/root` – root user home
- `/tmp` – temporary files
- `/var` – variable data (spool, logs, admin)
- `/usr/bin`, `/usr/lib`, `/usr/sbin` – user programs/libs/sys progs

Pseudo / Virtual Filesystems

System & kernel info exposed as files that **do not exist on real storage** – created/updated on the fly by the kernel.

- `/proc` – OS info: processes, memory parameters, ...
- `/sys` – system as a set of devices & buses
- Hierarchical; an interface to the kernel, but **not the only** way to do system calls

Info in /proc

- `/proc/<pid>/cmdline` – start command
- `.../cwd` – working dir (link)
- `.../environ` – env variables
- `.../exe` – running program (link)
- `.../fd/` – open files
- `.../mem`, `.../stat`, `.../statm`
- `/proc/device-tree` – device tree

Device Files /dev (slide 17)

b=block, c=char device (`ls -l /dev`). Write to a serial port:

```c
FILE *fp;
fp = fopen("/dev/ttyS0", "rw");
fprintf(fp, "Hello");
fclose(fp);
```

(Access typically needs sudo.)

NFS Mount Points – Lab2–13 (slides 19–20)

Root FS & home served over NFS from the host:

```
192.168.0.100:/srv/rpi_rootfs     -> /
192.168.0.100:/home/mc2/board     -> /home/root
# on target:
root@raspberrypi4-64:~# df -h
Filesystem                  Size  Used Avail Use% Mounted on
192.168.0.100:/srv/rpi_rootfs  982M 779M  133M  86% /
devtmpfs                    666M     0  666M   0% /dev
192.168.0.100:/home/mc2/board   80G  36G   41G  47% /home/root
```

Toolchain for Embedded Linux

Host vs. Target

- **Host** – development PC running Linux
- **Target** – the embedded system under development
- Connected via: **serial** (almost always, debugging), Ethernet (frequent), JTAG (low-level debug)

Native vs. Cross Toolchain

- **Native**: runs on workstation, generates code for the same machine (usually x86)
- **Cross-compiling**: runs on workstation (x86), generates code for the target (e.g. ARM)
- Native-on-target: runs on & builds for the target

Embedded Linux Solutions

- **Commercial** (own tools, open + proprietary mix): MontaVista, Wind River, TimeSys
- **Open source**, community-supported → **used in this course**
- Host PC: any recent Linux desktop; course uses **Fedora**

The 4 Toolchain Components (slide 34)

1. **GCC** (GNU Compiler Collection) – C/C++/Ada/Fortran/..., 60+ platforms; GPL (libs LGPL)
2. **Binutils** – generate/manipulate binaries: `as` (assembler), `ld` (linker), `ar`/`ranlib` (archives), `objdump`/`readelf`/`nm`/`size`/`strings` (inspect), `strip` (shrink)
3. **Kernel source & headers** – syscalls, constants, structs; under `<linux/...>`, `<asm/...>`; needed to build the C library
4. **C library** – interface kernel ↔ apps, provides standard C API (`printf...`); chosen when GCC is compiled

C Libraries & BusyBox

C Libraries

- **glibc** – standard GNU/Linux, perf+portable (LGPL); big: ~1.7 MB on ARM
- **uClibc** – lightweight for embedded (LGPL), ~400 KB
- **musl** – ~400 KB, MIT
- **dietlibc** – ~185 KB, GPL (stopped 2024)

BusyBox

The **SSwiss army knife"**: most CLI utilities in a **single executable** (even a web server!).

- <1 MB (static, glibc), <500 KB (static, uClibc)
- ls, cp, mount, grep, vi, wget, init, ... (hundreds)

"Hello WorldSSize Comparison (slide 44)

	static	dynamic	dyn. library
uClibc	18 kB	2.5 kB	400 kB
glibc	361 kB	2.7 kB	1.7 MB

⇒ uClibc wins massively when **statically** linked.

Building a Minimalistic Linux System

Build the Smallest Possible Linux

Step 1 – Kernel (from `kernel.org`):

```
make allnoconfig          # reset config to minimal
make menuconfig           # add only needed features
make                      # -> arch/x86/boot/bzImage
```

Step 2 – Root FS image (1 inode/1024 B → ~400 files):

```
dd if=/dev/zero of=rootfs.img bs=1k count=400
mkfs.ext2 -i 1024 -F rootfs.img
```

Step 3 – BusyBox + uClibc (static, cross-compiled → ~250k exe):

```
make menuconfig           # static, uClibc toolchain
make
make install              # into _install/
```

Step 4 – Populate & flush the FS:

```
mkdir /mnt/rootfs
mount -o loop rootfs.img /mnt/rootfs
rsync -a busybox/_install/ /mnt/rootfs/
chown -R root:root /mnt/rootfs/
sync
```

conclusion

Test without flashing hardware by booting in **QEMU** (next box).

Boot the System in QEMU (slide 51)

```
qemu \
  -m 32 \                          # RAM (MB) of emulated target
  -hda rootfs.img \                # emulated hard disk
  -kernel arch/x86/boot/bzImage \  # kernel image
  -append "root=/dev/hda"          # kernel command line
```

Bootloaders

Bootloader Software that runs first and is responsible for:

- basic hardware initialization
- loading the OS kernel (flash / network / non-volatile storage)
- optionally decompressing the kernel, then **starting** it
- usually a minimal shell (load data, inspect memory, diagnostics)

x86 Bootloaders

- **GRUB** – most powerful; reads many FS, network boot, shell
- **Syslinux** – network & removable media (USB, CD-ROM)

Embedded Bootloaders

- **U-Boot** (Denx) – de-facto standard (ARM, PPC, MIPS, x86...)
- **Barebox** – successor of U-Boot, cleaner, less HW support
- RedBoot, Yaboot, PMON (minor)

Raspberry Pi 4 Boot Process (slides 56–57)

BCM2711 powers up; Boot-ROM loads the main bootloader from EEPROM, which uses `BOOT_ORDER` (SD / network / USB). Stages:

1. **GPU 1st-stage** (ROM, fixed) – mounts FAT32 boot partition
2. **GPU 2nd-stage** (EEPROM) – loads/programs GPU firmware, restarts GPU
3. **GPU firmware** (`start.elf`) – configures SDRAM, starts CPU
4. **CPU** – runs `kernel8.img` (Linux) or 3rd-stage (U-Boot)

Boot methods: **Method 1** SD-card (Lab1), **Method 2** network/NFS (Lab2).

Inspecting & Programming the System

Inspect a Running System via /proc (slide 59)

```
cat /proc/version            # kernel version
cat /proc/cpuinfo            # CPU details
grep Free /proc/meminfo      # free memory
```

File Handling in C

- Open/close: `fopen()`, `fclose()`
- Write: `fputs()`, `fputc()`, `fprintf()`, `fwrite()` (binary)
- Read: `fgets()`, `fgetc()`, `fscanf()`, `fread()` (binary)

File Handling Example (slide 62)

```
#include <stdio.h>
void display_file() {
FILE *fp;
char str[100];
fp = fopen("file.txt", "rw");   // open for read/write
if (fp == NULL) {
printf("Error opening file\n");
return;
}
if (fgets(str, 100, fp) != NULL)
printf("%s\n", str);
fclose(fp);
}
```

Command-Line Interface mmymenu"(slide 60)

```c
int main(void) {
char line[100];
int a1 = 0, a2 = 0; char b[100];
while (1) {
printf("Command > ");
fgets(line, 100, stdin);
if (line[0] == 'x') return 0;      // exit
switch (line[0]) {
case 'a':                          // read 2 ints
sscanf(&line[1], "%d %d", &a1, &a2);
command1(a1, a2); break;
case 'b':                          // read string
sscanf(&line[1], "%s", b);
command2(b); break;
default:
printf("Unknown command '%c'\n", line[0]);
}
}
}
```

03 Linux GPIO

GPIO Registers (BCM2711)

GPIO & its Registers The BCM2711 provides **58 GPIOs**. Each pin can be GPIO or an **alternate function** (UART, I2C, SPI, ...). Controlled via register banks:

- **GPFSEL** – function select **and** direction (in/out/alt)
- **GPSET** – set output high, **GPCLR** – set output low
- **GPLEV** – read input level (high/low)

GPFSEL – Function Select 3 bits per pin, **10 GPIOs per 32-bit register** (GPFSEL0...5).

- 000 input, 001 output
- 100/101/110/111 alt fn 0–3, 011/010 alt fn 4–5

LSB of a pin's field:

```
#define FSELX_OFFSET(pin) (pin % 10 * 3)  // 3 bits per GPIO
```

SET / CLR / LEV Each is a bitfield, 1 bit per GPIO (banks 0 & 1 cover 0–57):

- write 1 to `GPSETn` bit → pin high
- write 1 to `GPCLRn` bit → pin low
- read `GPLEVn` bit → current input level

Register Offsets (base 0x7e200000)

0x00..0x14	GPFSEL0...5
0x1c / 0x20	GPSET0 / GPSET1
0x28 / 0x2c	GPCLR0 / GPCLR1
0x34 / 0x38	GPLEV0 / GPLEV1

All accesses are 32-bit.

GPIO Access Methods

Two Ways to Reach the GPIO Registers

- **Method 1 – Pseudo filesystem** (`/sys/class/gpio`): needs a GPIO driver, **portable**, less hassle with addresses, but **slow**
- **Method 2 – Memory mapped** (`/dev/mem` + `mmap`): direct pointer access to peripheral addresses, need the exact address, but **much faster**

Exam Matching: GPIO access methods (slide 14) Match each description (left) to the right term:

1. Page-Offset
2. Translates a physical address into a virtual one
3. Where the physical address of the GPIO register is declared
4. LED toggled via file manipulation under `/dev/mem`
5. LED toggled via file manipulation under `/sys/class/gpio`

Terms: **size of the virtual MMU page** · `mmap(...)` · Device Tree · Memory-Map manipulation · Pseudo-FS manipulation

Matches:

1. → size of the virtual MMU page
2. → `mmap(...)`
3. → Device Tree
4. → GPIO manipulation via **Memory Map**
5. → GPIO manipulation via **Pseudo File System**

Method 1: Pseudo Filesystem (sysfs)

sysfs GPIO File Structure Kernel config: **Device drivers → GPIO Support → sysfs interface**.

- `/sys/class/gpio/export`, `.../unexport` – (de)activate a GPIO
- `.../gpioXX/direction` – `in" or `out"
- `.../gpioXX/value` – `"1" or `"0"
- `.../gpioXX/edge` – `none/rising/falling/both`

Mind the GPIO Offset! (slide 18–19) On the InES RPi-Hat: LED4=GPIO12, LED5=GPIO13, Key0=GPIO22, Key1=GPIO23. But sysfs adds a **base offset of 512** (`gpiochip512`, label `pinctrl-bcm2711`). So in sysfs: **sysfs_number = 512 + pin** (e.g. GPIO12 → 524, GPIO22 → 534).

Control a GPIO via sysfs **LED (output), pin 12 → 524:**

```
echo 524 > /sys/class/gpio/export
echo out > /sys/class/gpio/gpio524/direction
echo 1   > /sys/class/gpio/gpio524/value    # on
echo 0   > /sys/class/gpio/gpio524/value    # off
echo 524 > /sys/class/gpio/unexport         # free
```

Button (input), pin 22 → 534:

```
echo 534 > /sys/class/gpio/export
echo in  > /sys/class/gpio/gpio534/direction
cat /sys/class/gpio/gpio534/value           # read
echo 534 > /sys/class/gpio/unexport
```

conclusion

Always add the **+512 offset** when converting a board GPIO number.

sysfs GPIO from C (slides 23–25)

```
// export a gpio
FILE *fp = fopen("/sys/class/gpio/export", "w");
fprintf(fp, "%i", gpio); fflush(fp); fclose(fp);

// set value
void fs_gpio_set(int gpio, int val) {
char str[100];
sprintf(str, "/sys/class/gpio/gpio%i/value", gpio);
FILE *fp = fopen(str, "w");
fprintf(fp, "%i", val); fflush(fp); fclose(fp);
}
// read value
int fs_gpio_get(int gpio) {
char str[100]; int ret;
sprintf(str, "/sys/class/gpio/gpio%i/value", gpio);
FILE *fp = fopen(str, "r");
fread(str, 101, 1, fp);
sscanf(str, "%i", &ret); fclose(fp);
return ret;
}
```

Method 2: Memory-Mapped GPIO

MMU & mmap The CPU sees **virtual** addresses; hardware lives at **physical** addresses. `/dev/mem` exposes physical memory; `mmap()` returns a **virtual** pointer to the (page-aligned) GPIO base. The program then works purely with virtual addresses; the MMU translates them.

```
void *mmap(addr, len, prot, flags, fd, base_addr);
```

Legacy Bridging The BCM2711 uses **35** address lines, but older peripherals still use the legacy **32**-bit map. A hardware **bridge** adapts new→old addresses; a `ranges` section in the **device tree** tells Linux about the mapping (e.g. ARM 0xFE200000 ↔ legacy 0x7E200000).

Memory-Mapped GPIO Access **Step 1 – map the base** (open `/dev/mem`, `mmap` one page):

```c
void *virtual_gpio_base;              // global pointer
int mmap_virtual_base() {
int m_mfd;
if ((m_mfd = open("/dev/mem", O_RDWR)) < 0)
return m_mfd;                         // FAIL
virtual_gpio_base = mmap(NULL, sysconf(_SC_PAGE_SIZE),
PROT_READ|PROT_WRITE, MAP_SHARED, m_mfd, GPIO_BASE_ADDR);
close(m_mfd);
if (virtual_gpio_base == MAP_FAILED) return errno;
return 0;
}
```

Step 2 – set/clear a pin (write the bit into SET0/CLR0):

```c
#define GPIO_SET0 0x1c
#define GPIO_CLR0 0x28
void mmap_gpio_set(int gpio, int value) {
uint32_t *reg = value
? (uint32_t*)(virtual_gpio_base + GPIO_SET0)
: (uint32_t*)(virtual_gpio_base + GPIO_CLR0);
*reg = (0x1 << gpio);
}
```

Step 3 – set direction (read-modify-write the FSEL field):

```c
#define GPIO_FSEL1 0x04
#define GPIO_FSEL2 0x08
void mmap_gpio_direction(int gpio, int dir) {
uint32_t *reg;
if (gpio <= 9)        reg = (uint32_t*) virtual_gpio_base;
else if (gpio < 20)   reg = (uint32_t*)(virtual_gpio_base + GPIO_FSEL1);
else                  reg = (uint32_t*)(virtual_gpio_base + GPIO_FSEL2);
*reg &= ~(0x7 << FSELX_OFFSET(gpio));   // clear field
*reg |=  (dir << FSELX_OFFSET(gpio));   // set direction
}
```

Device Tree

Device Tree – Purpose A data structure describing the **hardware** to Linux (especially **non-discoverable** hardware). Tells the kernel:

- physical addresses & sizes of memory/peripheral registers
- where interrupts and GPIO/peripheral pins are connected
- which alternate functions and **which driver** to load (+ enable it)

Principle & Files Non-discoverable HW is either coded in C in the kernel **or** described in a device tree:

- `.dts` – device tree **source**
- `.dtb` – compiled **blob** used by the kernel
- `dtc` – device tree compiler

The bootloader loads **both** kernel image and `.dtb` before starting.

Mechanism

1. Device tree is **parsed**
2. a **compatible** driver in the kernel is searched & started
3. the driver gets additional **properties** from the device tree

Structure Tree of `node@unit-address { property = value; ...};` nodes can hold child nodes; a **phandle** (`<&label>`) references another node. Values: strings, `<cells>` (32-bit), or `[bytes]`.

Exam Quiz: Device Tree (slide 42) **Q1 – What is a device tree?**

1. Definition of the processor architecture
2. Data structure about hardware properties
3. File structure of the filesystem (tree)
4. Holds info about hardware that can't be auto-detected

Q2 – What applies to the term device tree?

1. The same compiled kernel can support different HW configurations
2. Holds the virtual addresses of the machine hardware
3. Holds the device drivers of the machine hardware
4. Holds a list & addresses of nnon-discoverable"hardware

Q1: (2) and (4). (1) wrong scope, (3) confuses it with the filesystem.
Q2: (1) and (4). (2) false – it holds **physical** addresses; (3) false – it **references** which driver, it doesn't contain drivers.

GPIO node in bcm2711-rpi-4b.dts (slide 47)

```
gpio0: gpio@0x7e200000 {
compatible = "brcm,bcm2711-gpio";  // -> find driver
reg = <0x7e200000 0xb4>;           // base addr, size
interrupts = <0x00 0x71 0x04 ...>;
gpio-controller; #gpio-cells = <2>;
interrupt-controller;
pinctrl-names = "default";
phandle = <0x0f>;
i2c1_gpio2 { brcm,pins = <0x02 0x03>;  // GPIO 2 & 3
brcm,function = <0x04>; };             // alt fn 0 = I2C
};
```

Exam Exercise: Read a Device Tree Node (slide 49)

```
i2c@7e804000 {
compatible = "brcm,bcm2711-i2c\0brcm,bcm2835-i2c";
reg = <0x7e804000 0x1000>;
interrupts = <0x00 0x75 0x04>;
status = "okay";
clock-frequency = <0x186a0>;
};
```

- Device? **I2C**
- Address? `0x7e804000` – **legacy** view (`0x7e...`)
- Size? `0x1000` bytes
- Compatible driver? `brcm,bcm2711-i2c` (the `\0` separates a 2nd fallback driver `bcm2835-i2c`)
- Driver enabled? **yes** (`status = ökay"`; `"disabled"` → not loaded at boot)

Reading a Device Tree Node

- node name before `@` = **device**; after `@` = address
- `reg = <addr size>` → register base + length
- address `0x7e...` = legacy view, `0xfe...` = ARM view
- `compatible` = driver name(s); `\0` separates fallbacks
- `status = ökay"` loaded, `"disabled"` not loaded

I2C Bus

I2C Protocol

- **7-bit** slave address + $R/\overline{W}$ bit, then **8-bit** data bytes
- each byte **ACK**'d (SDA low = ACK, high = NACK)
- **S** = start condition, **P** = stop condition

MCP23008 I2C Expander Key registers:

- `IODIR (0x00)` – 0 = output, 1 = input
- `GPIO (0x09)` – read pin levels / write outputs
- `OLAT (0x0A)` – output latch
- `GPPU (0x06)` – pull-ups

sysfs maps its GPIOs as **GPIO578...585**.

Inspect & Drive I2C with i2c-tools

```
lsmod                          # list active drivers
rmmod pinctrl_mcp23s08_i2c     # free bus if a driver holds it
i2cdetect -l                   # list buses (e.g. i2c-1)
i2cdetect -y 1                 # scan bus 1 (UU=busy, 20=device@0x20)
i2cdump  -y 1 0x20             # dump all regs of device 0x20
i2cset   -y 1 0x20 0x0 0xf0 # write reg 0x00 = 0xf0 (IODIR)
i2cset   -y 1 0x20 0xa 0x0f # write reg 0x0a = 0x0f (OLAT)
```

Syntax: `i2cset [-y] bus chip-addr data-addr [value]`.

I2C-dev (C API)

Access an I2C Device via i2c-dev

Step 1 – open the bus and select the slave:

```
#include <linux/i2c-dev.h>
#include <sys/ioctl.h>
int fd = open("/dev/i2c-1", O_RDWR);      // bus file
if (fd < 0) { perror("open i2c"); exit(1); }
int addr = 0x20;
if (ioctl(fd, I2C_SLAVE, addr) < 0)       // pick slave addr
{ printf("no slave\n"); exit(1); }
```

Step 2 – write (register address + data):

```
uint8_t wr_buf[2] = { reg_addr, reg_data };
if (write(fd, wr_buf, 2) != 2) printf("write failed\n");
```

Step 3 – read (write the reg address, then read the data byte):

```
uint8_t wr_buf[1] = { reg_addr }, rd_buf[1];
write(fd, wr_buf, 1);                     // point to register
if (read(fd, rd_buf, 1) == 1) data = rd_buf[0];
```

Note: the bus cannot be opened if another driver already occupies it.

Lab 3–4 goal: test file-based GPIO and include it in the CLI, program memory-mapped GPIO with `mmap()`, and access the MCP23008 I2C expander both with i2c-tools and with i2c-dev.

Sample Exam (2019)

Exam: GPIO via Device Tree & sysfs (Task 3)

- driver name? → `brcm,bcm2711-gpio`
- size of all GPIO registers? → `0xb4` (from `reg`)
- find the gpiochip number for sysfs? → `ls gpiochip*/device/driver`
- switch on the LED at GPIO19 (= sysfs nr **340**):

```
echo 340 > /sys/class/gpio/export
echo out > /sys/class/gpio/gpio340/direction
echo 1   > /sys/class/gpio/gpio340/value
```

- mmap the GPIO_LVL register – base address: legacy `0x7e200000` / ARM-view `0xfe200000`
- value for the code's `#define`: `GPIO_BASE_ADDR = 0xfe200000`

Exam Exercise (Probeprüfung 2021)

Exam 2021 A3: GPIO access

A device-tree GPIO node is given; access the GPIOs.

- **a)** driver name: `brcm,bcm2711-gpio`
- **b)** register block size: `0xb4` bytes
- **c)** find the gpiochip nr: `ls gpiochip*/device/driver`
- **d)** LED at GPIO19 = sysfs nr **340**:
  ```
  echo 340 > /sys/class/gpio/export
  echo out > /sys/class/gpio/gpio340/direction
  echo 1 > /sys/class/gpio/gpio340/value
  ```
- **e)** GPIO base addr: Legacy `0x7e200000`, ARM-View `0xfe200000`
- **f)** `#define GPIO_BASE_ADDR 0xfe200000`
- **g)** `gpio_reg` (level reg, offset `0x34`) with `virtual_gpio_base = 0x84ec0000` → `0x84ec0034`

04 Linux Kernel

Kernel Characteristics & History

Linux Kernel – Characteristics
- An **executable binary**
- Drivers **can be part** of the kernel
- Must be **recompiled** if a driver is added as built-in (*)
- Linux is **both modular and monolithic**

Exam Quiz: What is true about the Linux kernel? (slide 2) Select one or more:
1. System calls are the only way to communicate with the kernel
2. Kernel modules run slower than built-in drivers
3. The kernel must be recompiled when a driver is added with *"
4. Drivers can be part of the kernel
5. It is an executable binary

Correct: (3), (4), (5).
- (1) false – sysfs//proc also communicate with the kernel
- (2) false – modules run at the **same** speed as built-in drivers

Birth of Linux
- **Stallman / GNU**: tools (gcc, bash), libs (glibc) – but **no kernel** (Hurd never finished)
- **Torvalds (1991)**: wrote a simple **monolithic** kernel LLinux", founded kernel.org
- Linux = GNU tools/libs + Torvalds' kernel

Exam Quiz: Components of the Linux project? (slide 4) (vs the GNU project) – mark the Linux-project components:
1. Free Software Foundation
2. Monolithic Kernel
3. Loadable Kernel Module
4. gcc
5. www.kernel.org

Correct: (2), (3), (5). FSF and gcc belong to **GNU** (Stallman), not the Linux project.

Kernel Architecture

Monolithic vs. Microkernel
- **Monolithic** (Linux): everything in kernel space; easier to program & debug
- **Microkernel** (GNU Hurd): only the essentials in the kernel; drivers/FS as user-space servers

Microkernel Core Components A microkernel keeps only:
- **Scheduler**
- **Interprocess Communication (IPC)**
- **Virtual Memory controller**

(File system & device drivers live in user space.)

Exam Quiz: Microkernel (Hurd) components? (slide 6) Select one or more: Scheduler · File System · IPC · Device Drivers · Virtual Memory Controller

Correct: Scheduler, IPC, Virtual Memory Controller. File System & Device Drivers are **user-space servers** in a microkernel.

Monolithic vs. Modular Kernel
- **Monolithic**: single executable, all drivers built in; adding a driver → **recompile** the whole kernel
- **Modular**: monolithic + a **module interface**; add a driver as a module **at run-time**, no recompile
- Linux is **both** (modular monolithic kernel)

Recent: PREEMPT_RT (real-time) merged into mainline since Linux **6.12** (Nov 2024); **Rust** declared no longer experimental (2025), now a core part of the kernel.

Getting & Building the Kernel

Getting the Sources From kernel.org via **HTTP / GIT / RSYNC**; the "vanilla"kernel. Release types:
- **mainline** (rc), **stable**, **longterm**
- tarballs (full source) + patches (diffs between versions)

```
git clone
    git://git.kernel.org/.../torvalds/linux
```

Source Tree – Key Directories
- init/ – arch-independent boot code (main.c → start_kernel())
- ipc/ – System V IPC (sem/msg/shm)
- kernel/ – core: sched, fork/exec/signal, modules
- arch/ – per-CPU ports (x86, arm, arm64, riscv...)
- drivers/ – **largest** part of the tree (~61%)
- fs/ – filesystems; include/ – headers (asm-*, linux)

Kernel Configuration Config stored in .config (text, key=value); never edit by hand:
- make menuconfig (text, needs libncurses-dev)
- make xconfig (graphical)
- undo: cp .config.old .config
- Yocto: bitbake -c menuconfig virtual/kernel

Built-in vs. Module
- * (**built-in**): compiled into the image, always available at boot, loaded by the bootloader
- M (**module**): loaded/unloaded **dynamically**, stored in the filesystem (needs FS access); easy driver dev: load, test, unload, rebuild...

Build & Install the Kernel **Host:**

```
make                          # build -> arch/<arch>/boot/{zImage,bzImage,Image}
make install                  # as root: installs /boot/vmlinuz-<ver>,
                              # System.map-<ver>, config-<ver>; updates grub
make INSTALL_MOD_PATH=<dir>/ modules_install   # default /lib/modules/<ver>/
```

Cross-compile (embedded): pass ARCH (= subdir in arch/) and CROSS_COMPILE (tool prefix):

```
make ARCH=arm CROSS_COMPILE=arm-linux-
```

Yocto handles all compiler switches:

```
bitbake virtual/kernel -c menuconfig   # configure
bitbake virtual/kernel                 # compile
```

Booting the Kernel

Boot with U-Boot
1. load zImage at address X
2. load <board>.dtb at address Y
3. start: bootz X - Y (ARM32) or booti X - Y (ARM64/RISC-V)

U-Boot passes the **DTB** alongside the kernel. Kernel command line:

```
console=ttyS0 root=/dev/mmcblk0p2 rootwait
```

(root= root FS, console= destination of kernel messages.)

First Process: /sbin/init First process, **ancestor of all**; controls runlevels:
- 0 shutdown, 1 single-user
- 2 multi-user (no NFS), 3 full multi-user
- 5 X11, 6 reboot

Runs startup/shutdown scripts per runlevel.

Shutdown Use /bin/shutdown – never pull the plug (buffered writes → corrupt FS). init sends **SIGTERM** then **SIGKILL**.
- -h halt, -r reboot, -p poweroff
- Ctrl-Alt-Del defined in /etc/inittab

System Calls

System Calls The main interface between **user space** and the **kernel** (~300 calls). They:

- make programming easier (kernel handles HW specifics)
- increase security (kernel checks each request)
- provide portability (stable interface, changing implementation)

Wrapped by the C library – **no direct access**.

Kernel Mode vs. User Mode

- **Kernel mode** (privileged): full, unrestricted HW access; any instruction/address; a crash **crashes the system**
- **User mode**: no direct HW/memory access (only via syscalls); crashes are **recoverable**; most code runs here

Exam Quiz: Kernel Mode (privileged)? (slide 33) Select one or more:

1. Allows access to all underlying hardware
2. The system can recover from program crashes
3. Reserved for the most safety-critical OS functions
4. Programs mostly run in privileged mode

Correct: (1) and (3). (2) false – a kernel-mode crash takes down the system; (4) false – most code runs in **user** mode.

Invoke a System Call on ARM Triggered by a **supervisor call** (software interrupt) `SVC #0`:

- `r7` = system call number (from `arch/arm/kernel/calls.S`)
- `r0-r2` = arguments
- `SVC #0` → switch to kernel mode, execute, return

Example – write "Hello Worldtto stdout (`sys_write` = 4):

```
HelloWorldString: .ascii "Hello World\n"
main:
mov r7, #4          /* sys_write */
mov r0, #1          /* stdout = 1 */
ldr r1, =HelloWorldString
mov r2, #12         /* string length */
svc #0
```

Modifying the Syscall Table One can in principle replace an entry in `sys_call_table[__NR_open]` with a custom function (e.g. spy code). **But** in modern kernels the table is in `.rodata` (read-only) – writing to it **instantly crashes** the system.

eBPF (extended Berkeley Packet Filter)

eBPF Runs **sandboxed** programs in kernel space **without** kernel modules or source changes.

- **In-kernel verifier**: static analysis → can't crash/hang the kernel
- **event-driven**: attach to syscalls, packets, kprobes
- **JIT compiled** (near-native), uses LLVM; **no loops**
- user ↔ kernel via eBPF **mmaps"**

eBPF with BCC in Python (slide 42)

```
from bcc import BPF
BPF_SOURCE_CODE = r"""
TRACEPOINT_PROBE(syscalls, sys_enter_mkdirat) {
bpf_trace_printk("Wow, new directory: %s\n", args->pathname);
return 0;
}
"""
bpf = BPF(text=BPF_SOURCE_CODE)
while True:                         # event loop
(task, pid, cpu, flags, ts, msg) = bpf.trace_fields()
print("%-6d %s" % (pid, msg))
```

System Logging

Kernel & System Messages

- `dmesg` – kernel message **ring buffer** (last ~200 msgs)
- `cat /var/log/messages` – kernel + system info via the syslog daemon

Console Log Levels

`KERN_EMERG`	0	system unstable / about to crash
`KERN_ALERT`	1	act immediately
`KERN_CRIT`	2	critical (HW/SW failure)
`KERN_ERR`	3	error (often driver HW issues)
`KERN_WARNING`	4	warning
`KERN_NOTICE`	5	notable (e.g. security)
`KERN_INFO`	6	informational
`KERN_DEBUG`	7	debug

Aliases in C: `pr_emerg`...`pr_debug`. View/set console level:

```
cat /proc/sys/kernel/printk   # current default minimum boot-default
echo 8 > /proc/sys/kernel/printk   # show all logs
```

05 Linux Kernel Module

Kernel Modules (LKM)

Loadable Kernel Module (LKM) A programming module that bridges user/application and hardware. Added **on the fly** while the OS runs (hence lloadable").

- Not a user program; minimal HW-access functionality
- Gateway to user space via a **virtual file** (`/sys/...`)
- Written in **C** only (no C++); Rust since Dec 2025
- Only **root** can load a module

Exam Quiz: Why a kernel driver instead of mmap()? (slide 4) Why implement a driver as a kernel driver if you could access peripheral addresses directly via `mmap()` from user space? Select one or more:

1. A kernel driver can be started automatically when the kernel loads the device tree
2. The GPL license requires using kernel drivers
3. For self-identifying hardware (USB, PCIe, HDMI) the driver must be in the kernel
4. A kernel driver gives faster access than `mmap()` from user space
5. In user space there are no direct interrupts from peripherals

Correct: (1), (3), (5). (2) false – no GPL requirement; (4) false – `mmap()` is **not** slower.

LKM Commands

- `insmod <module.ko>` – insert
- `rmmod <module>` – remove
- `lsmod` – list loaded modules
- `modinfo <module.ko>` – module info
- `modprobe` – insert/remove from `/lib/modules`

Module Types & Minimum Types: device drivers, filesystem drivers, network drivers. Minimum = an `__init` and an `__exit` method:

```
int  __init init_module(void)    { /* init */ }
void __exit cleanup_module(void) { /* clean shutdown */ }
```

Step 1: Simple Kernel Module

Module Skeleton

- `static int __init my_init(void)`
- `static void __exit my_exit(void)`
- register: `module_init(...)`, `module_exit(...)`
- info macros: `MODULE_LICENSE/AUTHOR/DESCRIPTION/`

Debugging: pr_* (no printf!) There is **no** `printf/stdio.h` in the kernel. `printk` is deprecated; use:

- `pr_info`, `pr_err`, `pr_alert`, `pr_emerg`
- `dev_info(dev, ...)`, `dev_err(dev, ...)`

Minimal Kernel Module (slide 13)

```c
#include <linux/kernel.h>
#include <linux/module.h>
static int __init sevenseg_init(void) {
pr_info("Hello from sevenseg driver!\n");
return 0;
}
static void __exit sevenseg_exit(void) {
pr_info("Bye from sevenseg driver!\n");
}
module_init(sevenseg_init);   // mandatory: identify init
module_exit(sevenseg_exit);   // and cleanup function
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Armin Weiss");
MODULE_DESCRIPTION("7 segment display driver");
MODULE_VERSION("1.0");
```

Tainted Kernel If `MODULE_LICENSE` is not a GPL-compatible tag, loading the module **taints** the kernel.

- `G`: all modules GPL(-compatible)
- `P`: a proprietary module/kernel is loaded
- bit 0 = proprietary, bit 10 = staging, bit 12 = out-of-tree module
- check: `cat /proc/sys/kernel/tainted`

Namespace Pollution Without symbol-visibility control, global names in different modules collide with the kernel and each other. **Always use `static` for global variables and functions!**

Step 2: Platform Driver

Discoverable vs. Non-Discoverable HW

- **Discoverable** (PCI, USB): device announces its identity on the bus
- **Non-discoverable** (I2C, SPI): must be declared explicitly in the **device tree**
- A kernel module talking to the device tree = **platform driver**

Platform Driver Responsibilities Defines a state struct + a `probe()` and `remove()` method.

- `probe(struct platform_device *pdev)`: init state, request device-tree info, map I/O memory, register IRQs/threads, register to a kernel framework
- `remove()`: undo the above
- register in `init()`, unregister in `exit()`

Platform Driver Definition & Registration (slides 24–25)

```c
static int sevenseg_probe(struct platform_device *pdev) {
pr_info("Hello from sevenseg_probe()\n"); return 0;
}
static int sevenseg_remove(struct platform_device *pdev) {
pr_info("Bye from sevenseg_remove()\n"); return 0;
}
static struct platform_driver sevenseg_driver = {
.driver = { .name = "sevenseg", .owner = THIS_MODULE },
.probe = sevenseg_probe,
.remove = sevenseg_remove,
};
static int __init sevenseg_init(void) {
return platform_driver_register(&sevenseg_driver);
}
static void __exit sevenseg_exit(void) {
platform_driver_unregister(&sevenseg_driver);
}
```

Step 3: Add the Device Tree

Matching a Device-Tree Node Provide a `compatible` list, export it with `MODULE_DEVICE_TABLE`, and reference it via `.of_match_table`. The driver then binds to the DT node with the same `compatible` string.

Device-Tree Match (slide 27)

```c
static const struct of_device_id sevenseg_ids[] = {
{ .compatible = "sevenseg" },
{ }
};
MODULE_DEVICE_TABLE(of, sevenseg_ids);
static struct platform_driver sevenseg_driver = {
.driver = { .name = "sevenseg", .owner = THIS_MODULE,
.of_match_table = sevenseg_ids },
.probe = sevenseg_probe, .remove = sevenseg_remove,
};
// device tree:  sevenseg { compatible = "sevenseg"; ... };
```

Step 4: Access the Hardware

Global Data Struct A struct shared between all methods, allocated with `devm_kzalloc` (auto-freed) and stored via `platform_set_drvdata`.

```
typedef struct sevenseg_platform_data {
struct device *dev;
void __iomem *base_addr;
int counter, mode, irq;
struct task_struct *kthread_sevenseg;
} sevenseg_platform_data;
// in probe():
pdata = devm_kzalloc(&pdev->dev, sizeof(*pdata), GFP_KERNEL);
platform_set_drvdata(pdev, pdata);
```

Access Peripheral Registers from the Driver **Step 1 – get the (virtual) base address** from the device tree:

```
res = platform_get_resource(pdev, IORESOURCE_MEM, 0);  // 35-bit phys
base_addr = devm_ioremap_resource(&pdev->dev, res);    // -> virtual
```

Step 2 – read/write (8/16/32-bit) at base_addr + offset:

```
void iowrite32(u32 value, void *addr);   // write
unsigned int ioread32(void *addr);       // read
```

The DT node carries `reg = <addr size>` and `interrupts = <...>`.

Step 5: Create a /sys File

Expose a sysfs Attribute **Step 1 – show/store callbacks** (read/write of the file):

```
static ssize_t counter_show(struct device *dev,
struct device_attribute *attr, char *buf) {
return scnprintf(buf, PAGE_SIZE, "value\n");   // user cat
}
static ssize_t counter_store(struct device *dev,
struct device_attribute *attr, const char *buf, size_t count) {
pr_info("New Message %s\n", buf);              // user echo
return count;
}
DEVICE_ATTR(counter, 0644, counter_show, counter_store);
```

Step 2 – create/remove the file in probe/remove:

```
device_create_file(&pdev->dev, &dev_attr_counter);   // in probe
device_remove_file(&pdev->dev, &dev_attr_counter);   // in remove
```

conclusion

File appears at
`/sys/bus/platform/drivers/<mod>/<PHYS_ADDR>.<mod>/counter`
e.g. `.../sevenseg/fe200000.sevenseg/counter` (→ `echo`/`cat`).

Interrupts

Interrupt Controller vs. Handler
- **Interrupt controller**: hardware that executes/routes the interrupt (ARM **GIC**)
- **Interrupt handler**: where the ISR is implemented (reacts)

Device-Tree Interrupt Definition `interrupts = <type number flags>`:
- type: 0 = SPI (shared), 1 = PPI (private)
- flags: 1 rising edge, 2 falling edge, 4 active-high level, 8 active-low level
- controller node: `compatible = ärm,gic-400"`, referenced via `interrupt-parent`

Calculate the GIC SPI Number (slide 47) For GPIO Bank 0 (→ `interrupts = <0x00 0x71 0x04>`):
1. **VideoCore IRQ**: GPIO Bank 0 (GPIOs 0–27) = IRQ **49**
2. **Hardware IRQ**: add ARM offset 96 → $49 + 96 = 145$
3. **GIC SPI number**: GIC reserves the first 32 (SGI/PPI), so subtract 32 → $145 - 32 = \mathbf{113} = 0x71$

So type 0x00 (SPI), number 0x71, flags 0x04 (active-high level).

Register an Interrupt Handler (slide 48)

```
static irq_handler_t irq_handler(unsigned int irq, void *dev_id) {
do_something();
iowrite32(0x1 << keys[1],
pdata->base_addr + GPEDSX_OFFSET(keys[1]));  // clear IRQ
return (irq_handler_t) IRQ_HANDLED;
}
// in probe():
pdata->irq = irq_of_parse_and_map(pdata->dev->of_node, 0);
devm_request_irq(pdata->dev, pdata->irq, irq_handler,
IRQF_TRIGGER_NONE, "irq_sevenseg", pdata);
```

Kernel Threads

kthread API
- `kthread_run(fn, data, namefmt)` – create + run, returns `task_struct` (name visible in `ps`)
- `kthread_should_stop()` – check inside the loop
- `kthread_stop(task)` – request stop

Kernel Thread (slide 52)

```
static int thread_fn(void *data) {
while (!kthread_should_stop()) {
do_something();
msleep(2000);
}
return 0;
}
// probe():  pdata->kthread = kthread_run(thread_fn, pdata, "kthread_7seg");
// remove(): kthread_stop(pdata->kthread);
```

Putting It Together (Lab 5–6)

7-Segment Counter combines all three feature types:
- **User requests** (write 2-digit numbers) → sysfs (`counter_show`/`counter_store`)
- **Hardware requests** (start/stop down-counting) → interrupt handler (`irq_handler`)
- **Iterative tasks** (periodic down-count) → kernel thread (`thread_fn`)

All glued by `sevenseg_probe`/`remove` + the `sevenseg_platform_data` struct.

Memory-Mapped I/O – Zero-Copy (optional) `mmap` maps the **same physical memory** into both kernel and user address spaces → **no copy**, no `read()`/`write()` syscalls for data transfer.
- driver: `remap_pfn_range()` maps physical pages into the user VM; set `VM_IO` (excludes from core dumps)
- user: `mmap(..., MAP_SHARED, fd, 0)` → changes visible to driver

Sample Exam (2019)

Exam: Complete the Platform Driver (Task 2) Fill the gaps so a '1' is written to the first GPIO address (from the device tree). Key fills:

```
static int my_module_probe(struct platform_device *pdev) {
int res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
void __iomem *base_addr = devm_ioremap_resource(&pdev->dev, res);
iowrite32(1, base_addr);                // write '1'
return 0;
}
static const struct of_device_id my_module_device_tree_ids[] = {
{ .compatible = "brcm,bcm2711-gpio" }, { }
};
MODULE_DEVICE_TABLE(of, my_module_device_tree_ids);
static struct platform_driver my_module_driver = {
.driver = { .name = "my_module", .owner = THIS_MODULE,
.of_match_table = my_module_device_tree_ids },
.probe = my_module_probe, .remove = my_module_remove,
};
static int __init my_module_init(void) {
return platform_driver_register(&my_module_driver); }
static void __exit my_module_exit(void) {
platform_driver_unregister(&my_module_driver); }
module_init(my_module_init);  module_exit(my_module_exit);
MODULE_LICENSE("GPL");
```

Exam Exercise (Probeprüfung 2021)

Exam 2021 A2: Complete the platform driver Fill in the module so it writes 1 to the first GPIO address from the device tree. Key fill-ins:

```
// in probe():
struct resource *res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
void __iomem *base_addr = devm_ioremap_resource(&pdev->dev, res);
iowrite32(1, base_addr);
// match table:
{ .compatible = "brcm,bcm2711-gpio", },
MODULE_DEVICE_TABLE(of, my_module_device_tree_ids);
// driver struct:
.of_match_table = my_module_device_tree_ids,
.probe = my_module_probe, .remove = my_module_remove,
// init / exit:
return platform_driver_register(&my_module_driver);
platform_driver_unregister(&my_module_driver);
module_init(my_module_init); module_exit(my_module_exit);
MODULE_LICENSE("GPL");
```

06 ARM Architectures

RISC vs. CISC

RISC vs. CISC

CISC	RISC
variable cycles/instr	~1 cycle/instr
many instr sizes/formats	one size & format
pipelining difficult	pipelining easy
memory-register arch	load/store arch
more addressing modes	fewer modes
fewer registers	more registers
complex cores	simple cores
simple compiler	complex compiler
emphasis on HW	emphasis on SW

Pipelining

Pipelining Principle Split instruction execution into stages (Cortex-M3: fe fetch, de decode, ex execute) and overlap them, like an assembly line.

- After the pipeline is filled, one instruction finishes **every** stage
- Requires: equal time per stage, no inter-stage dependencies
- Stage count is a design choice (typically 2–12)

Throughput (Instructions per Second)

- without pipelining: $\frac{1}{\text{instruction delay}}$
- with pipelining: $\frac{1}{\text{max stage delay}}$

Cortex-M3 (stages 3/4/5 ns, sum 12 ns):

$$\frac{1}{12\,\text{ns}} = 83.3\ \text{MIPS} \quad \rightarrow \quad \frac{1}{5\,\text{ns}} = 200\ \text{MIPS}$$

CPI (Cycles per Instruction)

- all-register ops → 6 instr in 6 cycles, CPI = 1
- LDR: read must finish on the bus before write-back (one write-back port) → 6 instr in 7 cycles, CPI = 1.2

Pros & Cons

+ big performance gain; simpler per-stage HW → higher clock
− every stage must handle every instr; all stages need memory access simultaneously

Pipeline Hazards

Three Hazard Types

- Control – branches: outcome unknown when the next instr must be fetched
- Data – an instr uses data modified in another stage: RAW (read-after-write, true dep), WAR (anti-dep), WAW (output dep)
- Structural – two instr need the same HW unit at once

Control Hazard – branch stall (slide 12) Conditional branch taken → 3 cycles to resolve (worst case):

```
1 CMP   R0, R1       ; R0 > R1 ?
2 BHI   go_on        ; if higher -> jump (outcome unknown at fetch)
3 MOVS  R3, #5       ; <- fetched but may need to be discarded (stall)
```

Data Hazard – RAW (slide 14) On a 4-stage pipe (fe/de/ex/wb); **not** on the 3-stage Cortex-M3:

```
1 ADDS R1, R2, R3    ; R2+R3 -> R1
2 ADDS R5, R4, R1    ; needs R1 before it
      is written back
```

Avoiding Structural Hazards Fetch multiple instructions at once and earlier than needed (Cortex-M3 fetches two 16-bit instr per transfer) → data bus free for other data next cycle.

In-order vs. Out-of-order Execution

- In-order: static scheduling; if one stalls, all stall; lower power; every CPU
- Out-of-order: dynamic scheduling; another instr runs while one stalls; only fetch & commit are in compiler order; higher power; needs a superscalar CPU. Issue in-order → execute out-of-order → commit in-order

ARM Cortex Profiles

The Three Profiles (post-2005)

- Cortex-A – Application: complex OS & user apps
- Cortex-R – Real-time: deterministic signal processing/control
- Cortex-M – Microcontroller: deeply embedded, optimized for cost/power

Cortex-M Profile

	M0+	M3	M4	M7	M23
arch	v6-M	v7-M	v7E-M	v7E-M	v8-M
MPU	y	y	y	y	y
DSP	n	n	y	y	n
FPU	n	n	y	y	n
pipeline	2	3	3	6	2
DMIPS/MHz	0.93	1.25	1.55	2.14	0.98

All use the Thumb-2 instruction set.

Cortex-R Real-time, deterministic; for safety-critical use (medical, aviation, automotive).

- Dual-core lock-step, fault detection via BIST
- ARM/Thumb-2, DSP + FPU, HW divider
- 8-stage pipeline, tightly-coupled memories (TCM), multi-level MPU
- low-latency interrupt controller

Cortex-A Application processors (ARMv8-A → ARMv9.2-A); 64-bit, big/LITTLE families.

- little (A53/A55/A520) vs big (A57/A72/A7x) cores
- ISA Thumb-2/ARM/A64; newest are A64-only
- pipeline 8... variable; in-order → out-of-order

ARM big.LITTLE

Three Operating Modes Pairs power-hungry "bigcores with efficient LLITTLEcores:

- Clustered switching: big & little in separate clusters; only **one cluster** active (high load → big); OS sees 4 CPUs
- In-kernel switcher: one big + one little form a virtual core; one real core powered at a time; OS sees 4 CPUs
- Heterogeneous MP (HMP): **all** cores can run at once; heavy/high-priority tasks → big, background → little; OS sees 8 CPUs

Xilinx Versal: combines scalar engines (Arm dual-core Cortex-R5F), adaptable engines (FPGA), and intelligent engines (AI/DSP), linked by a network-on-chip.

Debugging

Invasive vs. Non-Invasive Debugging

- Invasive: stop/step, breakpoints, read/write memory & registers, change ROM code (flash patch)
- Non-invasive (tracing): observe while running normally – real-time recording via ETM (Embedded Trace Macro), data recording, inserted `printf` messages

ICE & SWD

- In-Circuit Emulator (ICE): special CPU version with extra control/monitor signals; HW need not be functional; real-time; head is type-specific
- Serial Wire Debug (SWD): only 2 wires (SWDIO + SWCLK); runs the JTAG protocol on top

GNU Debugger (GDB) Stallman 1986, GPL, modelled after DBX.

- runs behind DDD, Eclipse, QT-Creator
- remote debugging via a debug server; not real-time

Compile & Debug with GDB Build with symbols (`-g`) and warnings (`-Wall`), then run with TUI:

```
gcc -g -Wall -o main main.c
gdb main -tui
```

Common commands: `run`/`r`, `print`/`p` **var**, `break`/`b` **line** (`... if cond` = conditional), `clear`, `next`/`n` (skip funcs), `step`/`s` (enter funcs), `continue`/`c`.

RISC-V

RISC-V Open **load/store** ISA, three base architectures **RV32I / RV64I / RV128I** (I = integer), module-based – the same instructions adapt across all three (128-bit set intentionally not yet finalized).

RISC-V vs. ARM/Intel

- base instructions: **47** (RISC-V) vs 1500+ (legacy x86)
- ISA: **modular** & incremental vs legacy
- license/royalties: **free & unlimited** vs $$$
- ecosystems growing rapidly (open & proprietary cores)

07 Scheduling

Processes & Threads

Process vs. Thread

- **Process**: a program in execution with its **own protected** address space (code/data/stack segments); one CPU runs one process at a time (context switching fakes concurrency); IPC is complicated
- **Thread**: llittle sisteröf a process; ≥1 per process, **shares** the process memory but has its own stack + registers
- Inter-thread comm is simpler (shared memory) but **no** memory protection between threads

Task States

- **Created** → admitted → **Ready**
- **Running**: executing on the CPU
- **Blocked**: waiting on a blocking condition
- **Preempted**: a higher-priority task interrupts → back to Ready
- **Terminated**: set inactive

Transitions: scheduler **dispatches** Ready→Running; block/wait Running→Blocked; preempt Running→Ready.

Linux Scheduling Policies

Scheduling Policies

Algorithm	Policy	RT
Completely Fair Sched.	`SCHED_OTHER`	no
First come first served	`SCHED_FIFO`	**yes**
Round Robin	`SCHED_RR`	**yes**
Batch	`SCHED_BATCH`	no
Run when nothing else	`SCHED_IDLE`	no

Completely Fair Scheduler (CFS)

Standard Linux scheduler (`SCHED_OTHER`) since kernel 2.6.23.

- distributes weighted runtime evenly; **vruntime** = runtime weighted by priority
- red-black tree sorted by vruntime; **leftmost** (least CPU) runs next
- timeslices typ. 4–24 ms; **not** for real-time (unpredictable)

Good vs. Bad Delay

- **Bad**: busy-wait `for(...){}` – task stays **running**, hogs the CPU
- **Good**: `usleep(...)` – task is **blocked**, other tasks may use the CPU

Real-Time Schedulers: FIFO & RR

SCHED_FIFO

- same priority: first task in the ready queue runs until it **completes or blocks** (no timeslice)
- different priority: a higher-priority task **preempts** a lower one

SCHED_RR

Like FIFO but each task has a fixed **timeslice** (`sched_rr_timeslice_ms`).

- runs for its slice **or** until blocked, then is preempted and re-queued (same priority)
- higher priority still preempts immediately

Exam Quiz: Which apply to Round Robin? (slide 22)

Select one or more:

1. An RR task can't be interrupted until its slice expires
2. A low-prio task can interrupt a higher-prio task if no low-prio task ran for a long time
3. Two low-prio tasks (e.g. both 20) alternate while higher-prio tasks (e.g. 40) are blocked
4. A higher-prio task can only preempt a lower one after an RR slice expires
5. All RR tasks have a slice; when it expires the task is interrupted by another **same-priority** task
6. If a task blocks, a ready **same-priority** task follows

Correct: (3), (5), (6). (1),(4) false – a higher-priority task preempts **immediately**; (2) false – RR has no priority aging/boost.

RR Response Time & Slice Choice

- active time < slice ⇒ **Response = Active Time**
- active time > slice ⇒ **Response = Active Time + Time Slice**

Slice too **small** → admin overhead dominates; too **large** → time-critical tasks locked out. **Optimal**: slightly > average task activity.

Get/Set the RR Timeslice

```
cat
     /proc/sys/kernel/sched_rr_timeslice_ms
          # read (default 100 ms)
echo '100' >
     /proc/sys/kernel/sched_rr_timeslice_ms
      # set
```

Exam Exercise: Scheduling Order (slide 11)

3 tasks on a 1-core CPU; each function: `usleep` (0.8 / 1.2 / 2.0 s) → LED**0/1/2** ON → ~3 s busy-loop → LED OFF. The number in the order = which LED toggles.

- **Option 1 – all** `SCHED_OTHER` (prio 40/30/20): CFS interleaves fairly → 0 1 2 0 1 2
- **Option 2 – all** `SCHED_FIFO`, prio f1 40 > f2 30 > f3 20: each runs to completion in priority order → 0 0 1 1 2 2
- **Option 3 – all** `SCHED_FIFO`, prio f1 20 < f2 30 < f3 40: each new (higher) task preempts; they finish LIFO → 0 1 2 2 1 0

Priority Ranges

Nice & RT Priorities

- **Nice** (SCHED_OTHER): −20 (highest) . . . +19 (lowest)
- **RT priority** (FIFO/RR): **1** (lowest) . . . **99** (highest)
- `top` columns: `PR` (priority), `NI` (nice); RT shown as `PR = rt` or negative

C-Program RT Value Offset

A real-time priority set in a C program is interpreted **+1** by the scheduler – e.g. setting 49 → scheduler uses 50.

RT Bandwidth Throttling

- `sched_rt_period_us` = 1 000 000 (100% CPU, default)
- `sched_rt_runtime_us` = 950 000 (95% for RT, default)

⇒ **5% of CPU is reserved** for non-RT (OTHER) tasks so RT tasks can't starve the system.

Command ttop"

Process status (`S`): `R` running, `S` sleeping, `D` uninterruptible sleep, `I` idle, `T` stopped, `Z` zombie. Columns: `PR/NI` (prio/nice), `VIRT/RES/SHR` (memory), `%CPU`, `%MEM`, `TIME+`, `COMMAND`. (`top -H` for threads not on BusyBox target.)

POSIX Threads

Create & Join Threads

Create – `pthread_create(&tid, attr, start_routine, arg)`: `attr`=NULL for defaults, returns 0 on success. **Join** – `pthread_join(tid, &ret)`: parent waits for the child (avoids **zombies**).

```c
#include <pthread.h>
void *print_msg(void *ptr) { printf("%s\n", (char*)ptr); return NULL; }
int main(void) {
pthread_t t1, t2;
const char *m1 = "Hello 1", *m2 = "Hello 2";
pthread_create(&t1, NULL, &print_msg, (void*)m1);
pthread_create(&t2, NULL, &print_msg, (void*)m2);
pthread_join(t1, NULL);
pthread_join(t2, NULL);
}
```

Compile: `gcc -o foo foo.c -lpthread`. Also: `pthread_exit`, `pthread_self`, `pthread_equal`, `pthread_detach` (no join needed).

Set Scheduler Policy/Priority in a Thread `sched_setscheduler(pid, policy, &param)`: `pid=0` = calling thread.

```c
#include <sched.h>
struct sched_param p;
p.sched_priority = 30;
sched_setscheduler(0, SCHED_FIFO, &p);   // policy + priority
```

`policy` ∈ {OTHER, RR, FIFO, BATCH, IDLE}.

Synchronization

Exam Exercise: Why Synchronize? (slide 47) `Task1`: `data = data*2` (T1.1); `print` (T1.2). `Task2`: `data = data/2` (T2.1); `print` (T2.2). Initial `data = 20`. What is printed? (data value → printed value)

- T1.1→40, T1.2→**40**, T2.1→20, T2.2→**20**
- T1.1→40, T2.1→20, T1.2→**20**, T2.2→**20**
- T2.1→10, T2.2→**10**, T1.1→20, T1.2→**20**
- T2.1→10, T1.1→20, T2.2→**20**, T1.2→**20**

Different interleavings give different output → **race condition** → need synchronization.

Mutex (Mutual Exclusion) Protect a critical section so only one thread enters at a time:

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_lock(&m);      // enter critical section
// ... shared data ...
pthread_mutex_unlock(&m);    // leave
sched_yield();               // relinquish CPU (-> end of prio queue)
```

Condition Variables For **waiting/signalling**; always paired with a locked mutex.

```c
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
pthread_mutex_lock(&m);
pthread_cond_wait(&c, &m);     // RELEASES m while waiting, re-locks on wake
pthread_mutex_unlock(&m);
// other thread:
pthread_cond_signal(&c);       // wake ONE waiter
pthread_cond_broadcast(&c);    // wake ALL waiters
```

No deadlock even if the signaller also locks `m`: `cond_wait` frees the mutex while blocked.

Deadlock & Priority Inversion

- **Deadlock**: A locks File1 and waits on File2, B locks File2 and waits on File1 → stuck forever
- **Priority inversion**: a low-prio task holds a lock a high-prio task needs, so the high-prio task waits behind it

Semaphores (Token Basket) `sem_post` adds a token, `sem_wait` takes one (blocks if empty).

```c
#include <semaphore.h>
sem_t s;
sem_init(&s, 0, 1);    // pshared=0, initial tokens=1
sem_wait(&s);          // take a token (block if none)
// ... critical work ...
sem_post(&s);          // return a token (wakes a waiter)
```

Also: `sem_getvalue`, `sem_destroy`.

Multiplex with Semaphores (slide 62) Two threads each blink an LED, guarded by one semaphore. Behaviour by initial token count:

- **1 token**: mutual exclusion – only one thread in the section at a time (alternating)
- **0 tokens**: nobody can enter – both block forever
- **2 tokens**: **both** threads run concurrently (no exclusion)

Rendezvous with Semaphores (slide 65) Two semaphores **both initialised with 0 tokens**; each thread posts the other's semaphore and waits on its own.

Forces the two threads to **meet at a sync point** before continuing – neither proceeds past the rendezvous until both have arrived. (Which LED turns off first depends on the post/wait order → test in the lab.)

Sample Exam (2019)

Exam: Thread Code Analysis (Task 4) 3 threads; each: lock mutex → `c = a++` (a starts 5) → set `SCHED_FIFO` prio = c → `cond_wait` → print `Output c : b++"` → unlock. `main`: `cond_broadcast` after 100 ms.

- a) **3** threads started
- b) priorities **5, 6, 7** → prio **7** is highest
- c) `SCHED_FIFO` = **preemptive** scheduling
- d) `cond_broadcast` wakes **all** waiting threads
- e) `pthread_join(tid[0])` waits only for the **first** thread → fix: loop `pthread_join(tid[i])` for all i
- f) output (highest prio first): `Output 7 : 0` / `Output 6 : 1` / `Output 5 : 2`

Exam: Scheduling Policy Comparison (Task 5)

	OTHER	R
priority	nice $-20..+19$	prio
highest	nice -20	prio
realtime?	no	y
preempt by	RT tasks + slices	higher RT
round-robin	variable slices	fixed
typical use	GUI	Audio

Exam Exercises (Probeprüfung 2021)

Exam 2021 A4: Threads + mutex/cond 3 threads; each: lock mutex, `c = a++` (a starts at 5), `param.sched_priority = c`, `SCHED_FIFO`, then `cond_wait`. `main` usleeps, then `cond_broadcast`.

- **a)** **3** threads started
- **b)** priorities **5, 6, 7** (the one with 7 is highest)
- **c)** `SCHED_FIFO` = **preemptive** scheduling
- **d)** `cond_broadcast` signals **all** threads to continue
- **e)** **No** – only `tid[0]` is joined; fix: loop `pthread_join(tid[i])` for all `M`
- **f)** output (highest prio first, `b++` post-increment):
  `Output 7 : 0` / `Output 6 : 1` / `Output 5 : 2`

Exam 2021 A5a: Scheduling policy comparison

	OTHER	
prio / nice	nice $-20..+19$	pri
best / worst	-20 hi / 19 lo	99 h
realtime?	no	
preempt by	RT + slices	higher
round robin	variable slices	fixe
typical use	GUI	audi

08 Realtime with Linux

Setting Priorities

Set Priority / Nice in Code **RT priority** (only `SCHED_FIFO`/`SCHED_RR`):

```
param.sched_priority = 10;
sched_setscheduler(0, SCHED_RR, &param);
```

Nice level (only `SCHED_OTHER`; no effect on RT threads):

```
setpriority(PRIO_PROCESS, getpid(), 19);
```

Set Priority from the Command Line

```
chrt -f 1 <command>         # run command as SCHED_FIFO prio 1
chrt -p -f 99 <pid>         # change running pid to FIFO prio 99
#  -o other  -f fifo  -r rr
nice -n 5 <command>         # start with niceness +5
renice -n -20 <pid>         # change niceness of a running process
```

Niceness: -20 (most favourable) $\ldots +19$ (least).

Real-Time A/V Example

Exam Example: A/V Scheduling Strategies (slides 10–15) 3 threads (Audio, Video, Control) together need >100% CPU. What happens under each policy?

- **All** `SCHED_OTHER`: nobody finishes in time; nice levels **unpredictable**. Audio loses samples (**highly perceptible**), video drops 6/30, control delayed
- **All** `SCHED_FIFO` (Audio P99 > Video P98 > Control P70): audio + video fine, but Control **never runs** (starvation)
- **Mixed** (Audio P99 RT; Video nice -5, Control nice $+5$, both OTHER): **graceful degradation** – audio perfect, video drops 2/30 (ok), control delayed 100 ms but acceptable

Interrupt Latency

Interrupt Latency Time from the external **interrupt** until the real-time handler **reacts**. Goal: keep it short & **predictable**.

Direct vs. Threaded IRQ Handler

- **Direct**: ISR + handler run in **interrupt context** → blocks **all** other tasks → keep as short as possible
- **Threaded** (modern): split in two
  - **Top-half** (`irq_handler`): disable HW interrupt, return `IRQ_WAKE_THREAD`
  - **Bottom-half** (`thread_handler`, a FIFO/RR thread): retrieve data, re-enable HW interrupt, return `IRQ_HANDLED`

Register a Threaded IRQ Handler

```
static irq_handler_t irq_handler(unsigned int irq, void *id) { // ISR
set_some_flags();
return IRQ_WAKE_THREAD;          // wake the thread handler
}
static irq_handler_t thread_handler(unsigned int irq, void *id) { // Task B
handle_interrupt();
return IRQ_HANDLED;
}
// in probe():
irq = irq_of_parse_and_map(pdev->dev.of_node, 0);   // from device tree
devm_request_threaded_irq(&pdev->dev, irq,
irq_handler, thread_handler, 0, "irq_sevenseg", NULL);
```

Latency During a System Call Ideal latency = the ISR time. **But**: if the interrupt occurs during a system call, by default Linux does **not** preempt the kernel – the syscall finishes first → latency becomes **unpredictable**. Solution: **kernel preemption**.

Kernel Preemption

Two Approaches to RT in Linux

- **Approach 1 – Fully preemptible kernel**: make the kernel preemptible with defined latencies → **PREEMPT_RT**
- **Approach 2 – Co-kernel**: a layer **below** Linux handles RT so Linux can't affect it → RTLinux, RTAI, **Xenomai**

PREEMPT_RT By Molnar, Gleixner, Rostedt. **Merged into mainline 20 Sep 2024** – kernel **6.12** is the first with built-in RT.

- Goal: **predictable, deterministic**, bounded **max latency** – **not** higher throughput (RT usually **lowers** it)
- `cyclictest`: default max ~1247 µs → RT max ~77 µs

PREEMPT_RT Techniques

- **Sleeping spinlocks**: waiters sleep instead of busy-looping
- **priority inheritance** for in-kernel locks
- **threaded IRQ** forced (unless `IRQF_NO_THREAD`)
- preemptible **RCU**, **high-resolution timers**

Preemption Models (kernel config)

- **No Forced Preemption (Server)**: syscalls never preempted; **max throughput**, fewest context switches
- **Voluntary (Desktop)**: preempt only at explicit **preemption points**
- **Preemptible (Low-Latency / Basic RT)**: most code preemptible (ms-range latency), **except** critical sections (spinlocks)
- **Fully Preemptible (Full RT)**: preempt everywhere except preempt-/interrupt-disable & `spin_lock`

Mutex vs. Spinlocks

Mutex (may block) Caller is **blocked** until the lock is released; the scheduler manages running/blocked. Context switches **add delay** → avoid for hard RT.

Exam Exercise: Does this software lock work on 2 CPUs? (slides 32–33)

```
while (lock == 1) {}    // wait until free
lock = 1;               // acquire
do_critical_code();
lock = 0;               // release
```

No – it has a race condition. tteständ ßetäre **two** instructions:

- CPU1 reads `lock==0`, before it writes `lock=1`, CPU2 also reads `lock==0`
- **both** set `lock=1` and enter the critical section

Fix: an **atomic** test-and-set (a `spin_lock`).

Spinlock – atomic busy-wait Atomic test-and-set; on ARM via `LDREX`/`STREX` (load/store exclusive).

```
spin_lock:
try_again: LDREX R2, [lock]    ; load exclusive
if R2 goto try_again           ; busy if already locked
STREX R2, 1, [lock]            ; store exclusive
if not R2 goto try_again       ; retry if store failed
```

No blocking, no scheduler, no context switch → useful with >1 CPU, fastest reaction. **But** kernel preemption is **disabled** while held.

Standard vs. Sleeping Spinlock

	`raw_spinlock_t`	`spinlock_t` (RT)
sleep?	no	**yes**
preemption	disabled	**enabled**
impl.	busy-wait	`rt_mutex`
latency / CPU	low / high	higher / lower

PREEMPT_RT turns most `spinlock_t` into **sleeping** spinlocks (`rt_mutex` with preemption), keeping only tiny/IRQ sections as raw.

Co-Kernel & CPU Affinity

Co-Kernel (Xenomai) Inserts an RT micro-kernel **between hardware and Linux** so RT tasks run independently of Linux. Generations: RTLinux (dead), RTAI (x86 only), **Xenomai**.

Pin a Thread to a CPU (Affinity) Reserve a core in a multi-core system:

```
cpu_set_t cpuset;
int cpu = 2;                          // CPU we want to use
CPU_ZERO(&cpuset);                    // clear the set
CPU_SET(cpu, &cpuset);                // add CPU 2
sched_setaffinity(0, sizeof(cpuset), &cpuset);   // 0 = this thread
while (1) {}                          // now burns only CPU 2
```

Reserve a CPU for IRQ handling: e.g. on a dual-core (Cyclone V, 2× ARM A9) pin all normal tasks to CPU0 and let the threaded IRQ handler own CPU1 → minimal, predictable interrupt latency.

Sample Exam (2019)

Exam: PREEMPT_RT vs Xenomai (Task 5b)

- **PREEMPT_RT**: make the (single) Linux kernel **fully preemptible**
- **Xenomai**: a **co-kernel** (micro-kernel) that does the scheduling between the real-time tasks and the normal Linux kernel

Exam Exercise (Probeprüfung 2021)

Exam 2021 A5b: PREEMPT_RT vs Xenomai Explain the difference in $\leq$3 sentences.

- **PREEMPT_RT**: make the (single) Linux kernel **fully preemptable**
- **Xenomai**: a **co-kernel** (micro-kernel) that handles the scheduling between real-time tasks and the normal Linux kernel

09 Linux Licenses

GPL (GNU General Public License)

GPL & the Four Freedoms A **copyleft** license: derivative works must be released under the **same** license, and the **source code must be available**. The four freedoms:

- **0** run the program for any purpose
- **1** study & modify it
- **2** redistribute copies
- **3** distribute your modified versions

Covers ~30–35% of free software (Linux kernel, BusyBox, MySQL).

Linux Kernel

- **Receive/buy** a Linux device → you get the sources + the right to study, modify, redistribute
- **Produce** a Linux device → you must release the sources to the recipient with the **same** rights, no restriction
- Programs linked against a GPL library must **also be GPL**

Tivoization → GPLv3 **Tivoization**: hardware only runs manufacturer-**signed** software, so modified open-source code can't run on the device. TiVo **complied** with GPLv2 (source available) but the modified builds were unusable → violated the **spirit** of copyleft. **GPLv3** (2007) adds: the user must be able to **run modified versions on the device** (+ explicit patent grant).

GPL: Must I Provide Source Code?

- **Not until you distribute** – keep modifications secret until product delivery
- When distributing a **binary**, do **one** of:
  - ship the source on a physical medium, **or**
  - give a network address where the source is found, **or**
  - include a **written offer valid 3 years** to fetch the source
- Don't want to provide source? → link against **LGPL** instead

LGPL (Lesser GPL)

LGPL Copyleft, but **lighter** – ~10% of projects (mostly libraries).

- modified versions of the library → same license
- a program **linked** against an LGPL library may stay **proprietary**
- **condition**: user must be able to **update the library independently** → use **dynamic linking**
- used by the C libraries (glibc, uClibc). Exceptions: MySQL (GPL or pay), Qt ≤4.4 GPL / ≥4.5 LGPLv3 or commercial

Licensing in Practice

Exam Example: What is your obligation? (slides 9–10)

1. Modify the Linux kernel / BusyBox / U-Boot (GPL)
2. Modify the C library or another LGPL library
3. Write an app that **uses** LGPL libraries (e.g. glibc)
4. Modify non-copyleft (permissive) software

- (1) GPL → **provide source code**
- (2) LGPL modified → **provide source code**
- (3) keep app **proprietary**, but **link dynamically** with the LGPL library
- (4) keep modifications proprietary, but **credit the authors**

Abusing kernel licensing: Nvidia puts a **wrapper** between its proprietary driver and the GPL kernel. Current trend: hide logic in **firmware or userspace** and keep the GPL kernel driver almost empty – it just blindly writes incoming binaries to hardware or exposes a large MMIO region to userspace via `mmap`.

Permissive Licenses

MIT & BSD "Do whatever you want, no obligation to the owner."

- permits use, redistribution, modification
- **source NOT required** to be distributed
- only requirement: keep the **copyright notice + warranty disclaimer** (include a license copy)
- MIT also explicitly allows merge, publish, sublicense, sell
- Apple uses **BSD** in macOS

MPL & Apache

- **MPL** (Mozilla, from Netscape 1998): **weak/file-level copyleft** – modified MPL **files** stay MPL, but a larger work can use other terms (e.g. Firefox)
- **Apache 2.0**: permissive + explicit **patent grant**; must state changes & keep notices; no source disclosure required

Summary Comparison

License Comparison

License	Type	Source?	Same licen
GPLv2/v3	strong copyleft	**yes**	whole worl
LGPLv3	weak copyleft	yes	library only
MPL 2.0	weak copyleft	yes	per file
MIT	permissive	**no**	–
BSD 2/3	permissive	no	–
Apache 2.0	permissive	no	– (+ pater

All require keeping the **license + copyright notice**; all disclaim liability & warranty. GPLv3/LGPLv3/Apache add an express **patent grant**; GPLv3/Apache also require **stating changes**.

Sample Exam (2019)

Exam: Licenses (Task 1)

- Sell an app **without** releasing source? → compile against an **LGPL** library (e.g. glibc) and **link dynamically**
- Sell a kernel driver without source? → **No** – the kernel is **GPL, not LGPL**
- `lsmod` shows `Tainted: P`? → a loaded module is **not declared GPL** (proprietary)

Exam Exercise (Probeprüfung 2021)

Exam 2021 A1: Licenses

1. Sell a Linux application **without** releasing the source – exact conditions (compilation type + library type)?
2. Sell a hardware **kernel driver** without source – possible?
3. `lsmod` shows `Tainted: P`. What does it mean?

- (1) compile against an **LGPL** library (e.g. glibc) and link it **dynamically**
- (2) **No** – the kernel is **GPL**, not LGPL
- (3) a loaded module is **not declared GPL** (proprietary)

10 GStreamer

Framework Overview

GStreamer A free, platform-independent **multimedia framework** (play, en-/decode, edit audio & video). Works on the **pipeline principle**: small modules chained into processing graphs. LGPL-2.1, 250+ plugins.

- **Element** (= plugin): one processing block
- **Pad**: an element's in/out connector (`src`/`sink`)
- **Pipeline** (top-level **bin**): the whole chain; **Source** → **Filter** → **Sink**

Kernel Audio/Video Interfaces

- **ALSA** (Advanced Linux Sound Architecture): the free sound layer in the kernel (by Kysela; replaced OSS in 2.6)
- **V4L** (Video4Linux): kernel drivers + API for real-time audio/video

Two Ways to Use GStreamer

- `gst-launch-1.0`: build/run pipelines from the **command line** – prototyping/debugging
- **GStreamer API**: build pipelines in **C** (`gst_*` calls)

ALSA (Audio)

Discover & Configure Audio Devices

```
aplay -l                                # list PLAYBACK devices (card/device #)
arecord -l                              # list CAPTURE devices
arecord --device=hw:1,0 --dump-hw-params  # show params of card1,dev0
alsamixer                               # interactive mixer (F3 play, F4 cap, F6 card)
amixer -c 1 scontrols                   # list simple controls of card 1
amixer -c 1 sget 'Master'               # read a control
amixer -c 1 sset 'Master' 80%           # set a control
```

Device string `hw:x,y` (x = sound-card #, y = device #); S16_LE = signed 16-bit little-endian.

V4L (Video)

Discover Video Devices & Formats

```
v4l2-ctl --list-devices                 # e.g. /dev/video0 (USB webcam)
v4l2-ctl -d /dev/video0 --list-formats-ext   # formats (YUYV, MJPG), sizes, fps
```

gst-launch-1.0 Prototyping

Build a Pipeline `gst-launch-1.0 [OPTIONS] PIPELINE-DESCRIPTION`

- chain elements with **!**
- set a property with `property=value` (applies to the **previous** element)
- inspect: `gst-inspect-1.0` (all elements), `gst-inspect-1.0 <element>` (its properties)

```
gst-launch-1.0 audiotestsrc ! audioconvert ! autoaudiosink
gst-launch-1.0 videotestsrc pattern=11 ! videoconvert ! autovideosink
```

Plugin Categories

- **Base**: small fixed core set, always maintained
- **Good**: good code + correct function, **LGPL**
- **Ugly**: good quality **but** license/patent redistribution issues
- **Bad**: not yet up to par (missing review/docs/tests/maintainer)

Key Elements

- `audiotestsrc` – test tone: `wave` (0 sine, 1 square, 2 saw, 3 triangle, 4 silence, 5 white-noise), `freq` (def 440, **max = rate/4**), `volume` (def 0.8, 0–1)
- `videotestsrc` – test pattern
- `autoaudiosrc`/`autoaudiosink` – auto-pick device
- `alsasink` – ALSA output: `device=hw:x,y`, sync
- `filesrc location=...` – read a file
- `audioconvert` – format/channel conversion
- `audioresample` – change sample rate
- `videoconvert` – colour-space conversion (RGB↔YUV)
- `videorate` – match a target framerate
- `v4l2src device=/dev/video0` – capture from a camera
- `jpegenc` – encode JPEG

Caps (Capabilities) A media-format filter inserted between elements: `media-type`, `format`, `rate`, `channels`, `width`, `height`. Written as a ppseudo-element:

```
gst-launch-1.0 audiotestsrc ! audioconvert ! \
  audio/x-raw,format=S8,channels=2 ! autoaudiosink
```

Video Processing Pipeline (slide 41)

```
gst-launch-1.0 v4l2src device=/dev/video0 \
  ! video/x-raw,framerate=30/1,width=640,height=480 \  # input format
  ! videorate ! video/x-raw,framerate=50/1 \           # -> 50 fps
  ! videoconvert ! video/x-raw,format=YUY2 \            # -> YUY2 colour
  ! jpegenc ! multifilesink location="color_img.jpg"    # color_img0000.jpg...
```

Useful plugins: `lame` (MP3 enc), `mpeg123` (MP3 dec), `videomixer`, `videocrop`, `wavenc`/`wavparse`, `videofilter` (bright/hue/flip/rotate), `audioecho`, `spectrum` (analyser), `equalizer-10bands`, `rtpsink`/`udpsink` (stream over network).

GStreamer C API

The 8-Step API Workflow

1. `gst_init(&argc, &argv)` – always first
2. `gst_pipeline_new(nname")` – create the bin
3. `gst_element_factory_make(ttype",nname")` + `g_object_set` (properties) – create & configure elements
4. `gst_bin_add_many(GST_BIN(bin), e1, e2, NULL)` – add to pipeline
5. `gst_element_link(a,b)` (or `_link_filtered` with caps) – link
6. `gst_element_set_state(bin, GST_STATE_PLAYING)` – play
7. `g_main_loop_new` + `g_main_loop_run` – run
8. cleanup: `g_main_loop_unref`, set state `GST_STATE_NULL`, `gst_object_unref(bin)`

Pipeline in C (steps 1–8)

```
gst_init(&argc, &argv);                                          // 1
GstElement *bin = gst_pipeline_new("test-pipeline");             // 2
GstElement *src  = gst_element_factory_make("audiotestsrc","source_1");
GstElement *sink = gst_element_factory_make("alsasink","sink_1");
g_object_set(G_OBJECT(src), "wave", 0, "freq", 1000.0, NULL);// 3 props
g_object_set(G_OBJECT(sink), "device", "hw:1,0", NULL);
gst_bin_add_many(GST_BIN(bin), src, sink, NULL);                 // 4
if (!gst_element_link(src, sink)) {                              // 5
g_printerr("Elements could not be linked.\n");
gst_object_unref(bin); return -1;
}
gst_element_set_state(bin, GST_STATE_PLAYING);                   // 6
GMainLoop *loop = g_main_loop_new(NULL, FALSE);                  // 7
g_main_loop_run(loop);
g_main_loop_unref(loop);                                         // 8
gst_element_set_state(bin, GST_STATE_NULL);
gst_object_unref(bin);
```

State Machine `NULL` (no resources) → `READY` → `PAUSED` (prerolling → prerolled) → `PLAYING`. Stop = back to `NULL`.

Caps Filter in C

```c
caps =
    gst_caps_new_simple("audio/x-raw",
"rate", G_TYPE_INT, 44100,
"channels", G_TYPE_INT, 2, NULL);
gst_element_link_filtered(conv, spect,
    caps);
gst_caps_unref(caps);
```

Shortcut: gst_parse_launch Replaces steps 2–5 with a single gst-launch-style string:

```c
bin = gst_parse_launch("audiotestsrc ! audioconvert ! autoaudiosink", NULL);
gst_element_set_state(bin, GST_STATE_PLAYING);
// ... main loop + cleanup as before
```

Message Handling (Bus Watch) Register a callback that receives every message on the pipeline's bus:

```c
bus = gst_element_get_bus(bin);
gst_bus_add_watch(bus, message_handler, &data);   // register callback
gst_object_unref(bus);
// callback:
gboolean message_handler(GstBus *bus, GstMessage *msg, gpointer data) {
g_print("From: %s, Type: %s\n",
GST_OBJECT_NAME(GST_MESSAGE_SRC(msg)),
GST_MESSAGE_TYPE_NAME(msg));
return TRUE;
}
```

Used e.g. with the `spectrum` element (`bands`, `threshold`, `post-messages`, `interval`) to read magnitude values per band.

11 OpenCV

Basics & Image I/O

OpenCV Open-source computer-vision library (image processing, machine vision, ML).

- Intel 1999 (Bradsky); C++/Python/Java; CUDA/OpenCL GPU
- **OpenCV-Python** = the Python API
- images are **NumPy ndarrays**: 2-D (grayscale) or 3-D (**BGR** colour), `uint8` 0–255

HighGui (no display on target!) HighGui (`imshow`, `namedWindow`, `createTrackbar`, `waitKey`...) needs **X-Windows**, which is **not** in the MC2 Yocto image → use `imwrite()` **instead of** `imshow()`.

Read / Write Images & Pass Arguments

```
#!/usr/bin/env python3              # shebang: which interpreter
import cv2, sys, numpy as np
path = sys.argv[1]                  # 1st command-line argument
img = cv2.imread(path, cv2.IMREAD_COLOR)   # ndarray, or None on failure
if img is None: sys.exit("Could not read the image.")
cv2.imwrite("out.png", img)         # type chosen from extension
```

imread flag: **1** `IMREAD_COLOR` (default), **0** `IMREAD_GRAYSCALE`, **-1** `IMREAD_UNCHANGED` (incl. alpha). Run: `chmod +x f.py ; ./f.py img.jpg`.

Capture Video

```
cap = cv2.VideoCapture(0)           # 0 = camera, or "file.webm"
if not cap.isOpened(): exit()
print(cap.get(cv2.CAP_PROP_FRAME_WIDTH))   # also FRAME_HEIGHT, FPS
while True:
ref, frame = cap.read()             # ref = success bool
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
cv2.imwrite("gray.jpg", gray)
cap.release()
```

Core Operations

Access & Manipulate Pixels

```
(h, w, c) = img.shape[:3]           # rows, cols, channels
img.dtype                           # e.g. uint8
img.size                            # h*w*c
px = img[100, 100]                  # [B, G, R] at y=100, x=100
blue = img[100, 100, 0]             # channel 0=B, 1=G, 2=R
img[100, 100, 2] = 0                # set red to 0
b, g, r = cv2.split(img)            # or faster: b = img[:,:,0]
img = cv2.merge((b, g, r))
img[:, :, 1] = 0                    # zero the green channel
```

Note: indexing is `img[y, x]` (row, col); NumPy slicing is **faster** than `split`/`merge`.

Drawing Primitives Each takes the image, geometry, **(B,G,R)** colour, then thickness (**-1 = fill**):

- `line(img, p0, p1, color, w)`
- `rectangle(img, c1, c2, color, t)`
- `circle(img, center, r, color, t)`
- `ellipse(img, c, (maj,min), angle, start, end, color, t)`

Borders & Timing `copyMakeBorder(src, top, bot, left, right, type, value)`; type = `BORDER_CONSTANT` / `REPLICATE` / `REFLECT`. Time a block:

```
t1 = cv2.getTickCount()
# ... code ...
dt =
     (cv2.getTickCount()-t1)/cv2.getTickFreq
```

Region of Interest (ROI) Slice `[start_row:end_row, start_col:end_col]` = `[y0:y1, x0:x1]`:

```
roi = img[70:100, 150:190]          # cut out (30 x 40 region)
img[10:40, 10:50] = roi             # paste it elsewhere
```

Convolution

Convolution **Element-wise multiply two matrices, then sum.** A small **kernel** slides left-to-right / top-to-bottom over the image; each output pixel = sum of (neighbourhood × kernel). Used for **blurring, sharpening, edge detection, Haar detection**.

Blur & Sharpen Kernels **Blur** (normalised average):

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Sharpen:

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Padding & Kernel Size

- **Odd** kernel sizes preferred (have a center pixel)
- a 3×3 kernel needs **1 line of padding**; generally `pad = (k-1)//2`
- padding methods: **replicate** (usual), zero, or wrap-around

Implement a Convolution

```
def convolve(image, kernel):
(iH, iW) = image.shape[:2]
(kH, kW) = kernel.shape[:2]
pad = (kW - 1) // 2                         # padding width
image = cv2.copyMakeBorder(image, pad, pad, pad, pad,
cv2.BORDER_REPLICATE)
output = np.zeros((iH, iW), dtype="float32")
for y in np.arange(pad, iH + pad):          # slide kernel
for x in np.arange(pad, iW + pad):
roi = image[y-pad:y+pad+1, x-pad:x+pad+1]  # neighbourhood
k = (roi * kernel).sum()                    # multiply + sum
output[y-pad, x-pad] = k
return output
# build a 5x5 blur kernel and apply to a grayscale image:
kernel = np.ones((5, 5), dtype="float") * (1.0/(5*5))
gray = cv2.cvtColor(cv2.imread(path), cv2.COLOR_BGR2GRAY)
cv2.imwrite("out.jpg", convolve(gray, kernel))
```

Built-in Filters (faster)

```
out = cv2.filter2D(gray, -1, kernel)    # ddepth -1 = same as src
g = cv2.GaussianBlur(img, (5,5), 0)     # ksize odd; sigma from size
a = cv2.blur(img, (5,5))                # average blur
m = cv2.medianBlur(img, 5)              # median blur
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)   # also BGR2RGB, BGR2YUV
```

Object Detection (Haar Cascades)

Haar Cascade Classifiers Object-detection using a **pretrained classifier in an XML file**.

- evaluate **Haar-like features** (edge/line/4-rectangle) on regions
- **AdaBoost** picks the most useful features, combining many **weak** classifiers into one **strong** classifier
- **cascade of stages** rejects obvious non-objects **early** → minimise the false-negative rate

Detect Faces / Objects

```
cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)   # must be grayscale
faces = cascade.detectMultiScale(gray, scaleFactor=1.1,
minNeighbors=3, minSize=(30,30), maxSize=(100,100),
flags=cv2.CASCADE_SCALE_IMAGE)
for (x, y, w, h) in faces:                       # returns rectangles
cv2.rectangle(image, (x,y), (x+w, y+h), (0,255,0), 2)
```

detectMultiScale Parameters

- **scaleFactor**: how much the image shrinks each pass. Small (e.g. 1.02) → fine search, accurate but **slow**; large (1.5) → fast but misses
- **minNeighbors**: overlapping rectangles needed for a hit. Small → more **false positives**; large → more **misses**
- **minSize/maxSize**: window size range to detect

OCR & Codes

pytesseract (OCR) Tesseract OCR engine: image → text (segmentation + recognition). `image_to_string(img, config=...)`.

- **PSM** (page seg): 6 single uniform block, 7 single line, 0 OSD only, 1 auto+OSD
- **OEM** (engine): 0 legacy, 1 LSTM neural net, 2 both, 3 default

Read QR / Barcodes (pyzbar)

```
from pyzbar import pyzbar
barcodes = pyzbar.decode(input)   # 1D + QR
for bc in barcodes:
x, y, w, h = bc.rect
info = bc.data.decode('utf-8')
cv2.rectangle(input, (x,y), (x+w,y+h), (0,255,0), 2)
```

License-Plate OCR (slides 59–60)

```
import cv2, pytesseract, numpy as np
gray = cv2.cvtColor(cv2.imread("car.jpg"), cv2.COLOR_BGR2GRAY)
plate_cascade = cv2.CascadeClassifier("haarcascade_russian_plate_number.xml")
plates = plate_cascade.detectMultiScale(gray, scaleFactor=1.1,
minNeighbors=5, minSize=(30,30), flags=cv2.CASCADE_SCALE_IMAGE)
for (x, y, w, h) in plates:
roi = gray[y:y+h, x:x+w]
kernel = np.ones((5,5), np.uint8)
roi = cv2.erode(roi, kernel, iterations=1)     # clean up
text = pytesseract.image_to_string(roi, config=r'--oem 3 --psm 6')
```

12 Embedded Machine Learning

AI / ML Foundations

AI ⊃ ML ⊃ DL ⊃ GenAI
- **AI**: emulate human behaviour (learn, decide, recognise)
- **ML**: algorithms that detect patterns in data (supervised / unsupervised)
- **DL**: ML using multi-layer **neural networks**
- **GenAI**: DL that **generates** content (text/images/code)

ML types: **task-driven** (supervised, predict next value), **data-driven** (unsupervised, find clusters), **reinforcement** (learn from mistakes).

Feed-Forward Neural Net Built from **neurons** in input / hidden / output layers, **fully connected**.
- knowledge stored in **weights** w and **bias** b
- can approximate **any continuous function**
- non-linearity from an **activation function**

Neuron Output & Parameter Count One neuron:

$$a = \sigma\left(\sum_i w_i a_i + b\right) = \sigma(w_1 a_1 + \cdots + w_n a_n + b)$$

The bias shifts the activation threshold. MNIST net (784→16→16→10):

$$\underbrace{784 \cdot 16 + 16 \cdot 16 + 16 \cdot 10}_{\text{weights}} + \underbrace{16 + 16 + 10}_{\text{biases}} = \mathbf{13\,002} \text{ parameters}$$

Learning = finding the right weights & biases.

Activation Functions

Activation Functions
- **Sigmoid**: $\sigma(z) = \dfrac{1}{1+e^{-z}}$ – öld school", not for hidden layers
- **Tanh**: $\dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$ – small gradient
- **ReLU**: $\max(0, z)$ – near-linear, easy to optimise, default choice
- **Leaky ReLU**: $\max(\alpha z, z)$ – small α avoids ReLU's zero-gradient for $z \leq 0$
- **Softmax** (output layer): values 0–1 summing to 1 → **probabilities**

Cost & Training

Cost & Gradient Descent **Cost** = how wrong the output is vs the label; **lower = better**. Training **minimises** the cost by walking downhill along its gradient. Risk: getting stuck in **local minima / saddle points**.

Cost / GD / Cross-Entropy Sum of squared differences:

$$C = \sum_k (\text{out}_k - \text{label}_k)^2$$

Gradient descent step (α = learning rate):

$$x \leftarrow x - \alpha \nabla f(x)$$

Cross-entropy loss (p true, q predicted):

$$H(p, q) = -\sum_x p(x) \log q(x)$$

Optimizers
- **Gradient Descent**: basic downhill step
- **Momentum**: adds a velocity term (smooths, escapes small dips)
- **RMSProp**: per-parameter adaptive step size
- **Adam** (Adaptive Moment Est.): **RMSProp + Momentum**, the usual default

Epoch / Batch / Iteration
- **Epoch**: one full pass of the **entire** dataset (fwd + back)
- **Batch size**: training examples per batch
- **Iterations**: batches needed for one epoch

1 Epoch = Iterations × Batch Size

Keras Workflow

Frameworks **Keras** – high-level API (runs on TensorFlow, `tf.keras`); **TensorFlow** (Google) – dataflow/symbolic math; **PyTorch** (Meta) – Torch-based. All written in Python.

Build / Compile / Train a Model

```
from tensorflow import keras
from tensorflow.keras import layers
model = keras.Sequential([                          # define
keras.Input(shape=input_shape),
layers.Flatten(),                                   # 28x28 -> 784
layers.Dense(16, activation="relu", name="Layer1"),
layers.Dense(16, activation="relu", name="Layer2"),
layers.Dense(num_classes, activation="softmax"), # 10 outputs
])
model.summary()
adam = keras.optimizers.Adam(learning_rate=0.005,        # compile
beta_1=0.9, beta_2=0.999, amsgrad=False)
model.compile(optimizer=adam,
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, batch_size=50, epochs=5, verbose=1)  # train
```

Other losses: `mse`, `mae`, `categorical_crossentropy`.

Load & Preprocess MNIST

```
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.astype("float32") / 255    # scale pixels to [0,1]
x_test  = x_test.astype("float32")  / 255
x_train = np.expand_dims(x_train, -1)        # (28,28) -> (28,28,1)
x_test  = np.expand_dims(x_test, -1)
```

60 000 training + 10 000 test images.

Deployment (TensorFlow Lite)

Convert to TFLite & Run Inference **Convert** (on host):

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
open("model.tflite", "wb").write(converter.convert())
```

Inference (on target): input = 28×28 floats (0..1), output = 10 probabilities; pick the `argmax`.

```
interpreter = Interpreter("model.tflite")
interpreter.allocate_tensors()
in_d  = interpreter.get_input_details()[0]
out_d = interpreter.get_output_details()[0]
def predict(image):
inp = np.array(image, dtype="f").reshape([1, 28, 28, 1])
interpreter.set_tensor(in_d["index"], inp)
interpreter.invoke()
out = interpreter.get_tensor(out_d["index"])[0]
return int(np.argmax(out))           # most probable digit
```

In C: `TfLiteModelCreateFromFile` → `...InterpreterCreate` → `AllocateTensors` → copy input → `...Invoke` → read output.

Convolutional Neural Networks

CNN

- **Convolution**: filter input with a kernel → **feature extraction** + data reduction
- **Pooling** (sub-sampling): max- or average-pool reduces dimensions & parameters → faster training, controls **over-fitting**

Conv / Pool Output Sizes Valid 5×5 conv: $n \to n-4$; 2×2 max-pool (stride 2): $n \to n/2$.

- 28 → `Conv5` 24 → `Pool2` 12
- 12 → `Conv5` 8 → `Pool2` 4
- flatten $4 \cdot 4 \cdot 32 = 512$ → Dense 10

Build a CNN in Keras

```
model = keras.Sequential([
keras.Input(shape=input_shape),
layers.Conv2D(16, kernel_size=(5,5), activation="relu"),
layers.MaxPooling2D(pool_size=(2,2)),
layers.Conv2D(32, kernel_size=(5,5), activation="relu"),
layers.MaxPooling2D(pool_size=(2,2)),
layers.Flatten(),
layers.Dense(num_classes, activation="softmax"),
])
# ~18'378 params, ~98.67% MNIST accuracy (vs dense-only net)
```

Image Acquisition for ML (Lab 7)

Webcam → 28×28 Pipeline Host trains → `model.tflite`; target grabs & preprocesses an image:

```
gst-launch-1.0 v4l2src device=/dev/video0 \
  ! video/x-raw,width=640,height=480 \
  ! videorate ! "video/x-raw,framerate=1/1" \     # 1 fps
  ! videobalance contrast=2.0 \                   # increase contrast
  ! videoconvert ! "video/x-raw,format=GRAY8" \   # greyscale
  ! videocrop top=0 left=80 right=80 bottom=0 \   # 640x480 -> 480x480
  ! videoscale ! "video/x-raw,height=28,width=28" \  # -> 28x28
  ! multifilesink location="image.raw"
```

Use `tee` to branch the stream (e.g. save both the 28×28 raw and a JPEG preview).

Beyond: GAN, Transformers, LLMs

GAN **Generative Adversarial Network**: a **generator** creates fake samples to fool a **discriminator**, which classifies real vs fake; both improve via back-propagation.

AI Evolution **FCN** (math functions) → **CNN** (2-D spatial features) → **Transformers** (sequence/context) → **LLMs** (reasoning at extreme scale).

LLMs (Transformer Workflow)

1. **Tokenization & embedding**: text → tokens → high-dimensional vectors
2. **Self-attention (QKV)**: Query/Key/Value vectors weigh how relevant surrounding tokens are
3. **Probability & softmax**: rank the vocabulary, pick the highest-probability next token

LLMs = **next-token prediction**; a text subset of GenAI, trained with billions of parameters.

13 Virtual Memory (MMU)

The Problem & the MMU

Logical vs. Physical Address Space
- **logical $\ll$ physical**: the address capacity is limited by the **architecture** (e.g. C8086: 640k usable) $\rightarrow$ expand it
- **logical $\gg$ physical**: **physical memory too small** $\rightarrow$ use disk as virtual memory (**swapping**)
- today (64-bit): the limit is the **physical memory**

The MMU The Memory Management Unit sits between CPU and memory and **translates logical (virtual) addresses to physical addresses** (LA$\rightarrow$PA), under OS control.

Memory Expansion (Historical)

Overlay & Bank Switching
- **Overlay**: load program phases from disk into one shared memory region as needed
- **Bank switching** (C8080): 16 address bits $\rightarrow$ 64 kByte. Lower 32 kByte **fixed** (`A15=0`); upper 32 kByte **paged** (`A15=1`) via a 2-bit **page register** $\rightarrow$ 4$\times$32 kByte banks

Segmentation

Fixed vs. Variable Partitions
- **Fixed** partitions: each process has a **base register**; a process switch triggers a page switch (OS)
- **Variable** (Intel 8086): physical addr = (16-bit segment register $\times$ 16) + 16-bit offset = **20-bit** physical; base = `n`$\times$`0x10`, freely placed
- a **limit register** bounds the segment $\rightarrow$ overflow = **protection fault**

8086 Segment Translation

$$\text{phys} = (\text{segment} \ll 4) + \text{offset}$$

e.g. segment `0x5A23`, offset `0x1234`: `0x5A230 + 0x1234 = 0x5B464`.

Paging

Paging Principle RAM is split into fixed-size **page frames** (typ. 4 kB); a process is split into equal-size **blocks** (block size = frame size). A per-process **page table** (in RAM) maps blocks $\rightarrow$ frames, so blocks need **not** be contiguous.

Address Translation Virtual address = **VPN** (virtual page nr) + **VPO** (offset); physical address = **PPN** (physical page nr) + **PPO**.
- the **VPN indexes** the page table (base in **PTBR**)
- **PPO = VPO** (offset unchanged by translation)
- **valid/present bit = 0** $\rightarrow$ **page fault** (not in RAM)

Demand Paging & Swapping
- **Demand paging**: load a block from disk only when needed; the **present bit** says whether it's in RAM
- **Swapping**: if RAM is full, a **replacement policy** (**FIFO / LRU / LFU**) picks a victim
- **Controlled write-back**: only write the victim to disk if its **dirty bit = 1**

Protection & Sharing
- **Protection bits** per entry: executable / read-only / user-accessible
- **Sharing**: one frame can map into several processes with **different rights** (e.g. `SUP` = kernel-only)

Dimensioning Formulas (critical!)

Core MMU Formulas Let page size P, virtual address = v bits, physical address = m bits.
- page offset bits: VPO = PPO = $\log_2 P$
- virtual page nr: VPN = v − VPO
- physical page nr: PPN = m − PPO
- **# page table entries** = $2^{\text{VPN}} = \dfrac{\text{virtual address space}}{P}$
- page table size = (# entries) $\times$ (entry width)

Dimensioning a Paging System
1. offset bits = $\log_2$(page size) (VPO = PPO)
2. VPN = (virtual addr bits) − VPO
3. PPN = (physical addr bits) − PPO
4. # entries = 2^{VPN}
5. page table size = $2^{\text{VPN}}\times$ entry width

Worked Examples & Exercises

Dimensioning Example (slide 39) 4 GByte virtual (32-bit), 16 MByte physical (24 addr lines), **block size 1 MByte**:

- page offset = $\log_2(1\text{M}) =$ **20** bit
- page number (VPN) = $32 - 20 =$ **12** bit $\rightarrow 2^{12} =$ **4096** table entries
- physical block nr (PPN) = $24 - 20 =$ **4** bit

Page Table Size (slide 42) 32-bit virtual, 4 kByte pages (offset 12), entry width ~19 bit:

VPN = $32 - 12 = 20 \rightarrow 2^{20}$ entries.

$$2^{20} \times 19\,\text{bit} = 19.9\,\text{Mbit} \approx \textbf{2.5 MByte}$$

(the page table itself lives in main memory).

Exercise 1 – Number of Page Table Entries (slide 41) # entries = 2^{VPN} = virtual address space/P:

Virt.	P	# entries
16-bit	4k	$2^{16}/2^{12} = 2^4 =$ **16**
16-bit	8k	$2^{16}/2^{13} = 2^3 =$ **8**
32-bit	4k	$2^{32}/2^{12} = 2^{20} =$ **1M**
32-bit	8k	$2^{32}/2^{13} = 2^{19} =$ **512k**

Exercise 2 – VPN / VPO / PPN / PPO (slide 53) 32-bit virtual, 24-bit physical. VPO = PPO = $\log_2 P$; VPN = 32−VPO; PPN = 24−PPO:

P	VPN	VPO	PPN	PPO
1 kB	22	10	14	10
2 kB	21	11	13	11
4 kB	20	12	12	12
8 kB	19	13	11	13

Exam Exercise – 1 kByte Pages (slide 54 / 2021 A8) MMU with **1 kByte** pages; page table entries are **22 bits** (no management bits). Virtual address space = **22-bit** (from the diagram).

- **a)** VPO = PPO = $\log_2(1\text{k}) = 10$ bit (VPO and PPO are always equal)
- **b)** entry = PPN = 22 bit, so physical addr = PPN + PPO = $22 + 10 = 32$ bit $\rightarrow 2^{32} =$ **4 GByte**
- **c)** VPN = $22 - 10 = 12 \rightarrow 2^{12} =$ **4096 entries**
- **d)** TLB bit width = page-table entry width = **22 bit**

Exam: mmap Virtual Address (Aufgabe 3g) PPO = 65 536 byte = 2^{16} $\rightarrow$ the lower **16 bits** are the page offset (preserved by translation). `GPIO_BASE_ADDR` is mapped to `virtual_gpio_base` = `0x84ec0000`; `gpio_reg` sits at register offset `0x34`.

The page offset (PPO = VPO) is unchanged by `mmap`, so:

$$\texttt{0x84ec0000} + \texttt{0x34} = \texttt{0x84ec0034}$$

User-Space Access: mmap

Memory-Mapped GPIO via mmap `/dev/mem` exposes physical memory; `mmap()` returns a **page-aligned** (4096) virtual pointer to the GPIO base.

```
void *virtual_gpio_base;                    // global pointer
int mmap_virtual_base() {
int m_mfd;
if ((m_mfd = open("/dev/mem", O_RDWR)) < 0) return m_mfd;
virtual_gpio_base = mmap(NULL, sysconf(_SC_PAGE_SIZE),
PROT_READ|PROT_WRITE, MAP_SHARED, m_mfd, GPIO_BASE_ADDR);
close(m_mfd);
if (virtual_gpio_base == MAP_FAILED) return errno;
return 0;
}
// access:  *(virtual_gpio_base + gpio_register_offset)
```

`GPIO_BASE_ADDR` (page-aligned) comes from the **device tree**.

Performance: Cache & TLB

MMU is Slow The page table lives in main memory, so each CPU access needs **two memory accesses**: (1) read the page table, (2) read the physical memory.

Virtual vs. Physical Cache

- **Virtual cache** (before MMU): fewer MMU accesses $\rightarrow$ **faster**; must cover the (larger) logical space; must be **invalidated on task switch**
- **Physical cache** (after MMU): **all** accesses slowed by the MMU

TLB **Translation Lookaside Buffer** = a **cache of the page table**.

- **TLB hit** $\rightarrow$ instant physical address
- **TLB miss** $\rightarrow$ walk the page table
- entry invalid $\rightarrow$ **page fault** (load from disk)

Finding the Page Table: PTBR / TTBR How does the CPU find the page table (which is **in** memory) without a lookup? A register holds its **physical** base address, set by the OS on each context switch:

- **PTBR** (x86: `CR3`)
- ARM Cortex: `TTBR0` (user) / `TTBR1` (kernel)

Caches also store the MMU's **access rights** (R/W, privilege) alongside the data, checked on every access (and for inter-process isolation).

Segmentation vs. Paging

- **Paging** +: no adder, equal size $\rightarrow$ easy swapping, transparent. −: longer lookups, page-table memory, internal fragmentation
- **Segmentation** +: smaller table, flexible size saves memory. −: unequal size hurts swapping, needs programmer effort, adder/comparator

Summary **Advantages**: contiguous appearance despite fragmentation; process **isolation**; shared maps for multi-threading; **fault isolation**; security. **Disadvantages**: slower (translation) + extra page-table memory. Access cost factors: **page fault**, **TLB miss**, **cache miss**.